

Opus Publica

RC2 Governance API Specification

The Programmable Interface of the Self-Enforcing Governance System

Version 1.0

Issued 09 August 2026

Document Type	Technical Documentation / API Reference (Implementation Spec IS-4)
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Integration Engineers, SDK Authors, CI/CD Engineers, External Auditors
Classification	Normative — RC2 API Contract
Builds On	RC2 Evolution Roadmap (MRC, ATG, CCE, Dashboard, SAP pillars)
API Specification Format	OpenAPI 3.1 — https://swagger.io/specification/
Effective Date	09 August 2026
Supersedes	None (first issuance of the Governance API Specification)
Review Cadence	On every Governance API minor or major version; quarterly consistency audit

Document ID	OP-RC2-API-001
-------------	----------------

The RC2 governance system is not a dashboard humans read; it is an API every tool — CI, dashboards, auditors, IDE plugins — consumes. This document is the contract for that API.

Chapter 1 — Opening Hook and Introduction

1.1 The Governance System as a Programmable Interface

The RC1 constitutional hierarchy of Opus Publica was, by design, a set of documents that humans read. The RC2 Roadmap converted those documents into a machine-readable constitution (MRC), an automated traceability graph (ATG), a continuous certification engine (CCE), a governance dashboard, and a self-auditing platform (SAP). What the RC2 Roadmap named as five pillars, this specification exposes as a single programmable surface: the Governance API. Every dashboard rendered, every CI check executed, every external auditor query answered, every IDE plugin inspection performed — all of them SHALL flow through the same set of HTTPS endpoints specified here. The thesis of this document is that a governance system that does not expose a stable, versioned, machine-readable API is not, in any meaningful sense, self-enforcing; it is merely a database with a UI, and a database with a UI cannot be consumed by a CI pipeline, a Backstage plugin, or a regulator's audit script.

The Governance API is the read-write surface of the governance system. Reads retrieve the current constitutional state (provisions, criteria, traceability chains, assessment results, SLO statuses, audit findings, release records). Writes record new evidence, request synchronous criterion evaluation, post remediations, authorize deployments, and register webhook subscriptions. Every read is auditable; every write is authenticated, authorized, idempotent, and immutable. The API is the contract by which the governance system ceases to be an internal tool and becomes a platform.

The API Prime Directive

Every governance primitive — provision, criterion, traceability node, evidence artifact, assessment result, SLO, finding, anomaly, release record — SHALL be addressable by a stable URI and retrievable by an authenticated HTTPS GET. Every governance mutation — evidence upload, sign-off, remediation, deployment authorization — SHALL be performable by an authenticated, idempotent HTTPS POST or PUT. There SHALL be no governance primitive reachable only through the UI, only through the database, or only through an internal library. The API is the surface; the UI is a consumer of the surface, not the surface itself.

1.2 Purpose, Audience, and Prerequisites

Purpose. This specification defines the contract that the RC2 Governance API exposes to all consumers. It is the normative reference for the OpenAPI 3.1 document, the SDKs, the CLI, and every integration built on top of the governance system. Where the OpenAPI document and this specification disagree, this specification SHALL prevail; the OpenAPI document SHALL be amended to align. This specification is binding on the governance system implementers and on every integration author.

Audience. The intended readers are: (1) integration engineers wiring CI/CD pipelines to the certification engine; (2) dashboard engineers building read-only visualisations on top of the API; (3) external auditors and regulators writing scripts to retrieve evidence; (4) SDK authors maintaining the Python, TypeScript, and Go client libraries; (5) IDE plugin authors surfacing traceability and certification status inside the developer's editor; (6) platform engineers operating the governance system itself.

Prerequisites. The reader SHALL be familiar with: HTTP semantics as specified in RFC 9110; the JSON data interchange format specified in RFC 8259; the OpenAPI 3.1 specification; OAuth 2.0 client credentials flow as specified in RFC 6749; JSON Web Tokens as specified in RFC 7519; and the RFC 7807 Problem Details for HTTP APIs error format. Familiarity with the Opus Publica RC2 Evolution Roadmap and the RC1 constitutional hierarchy is assumed.

1.3 Scope

This specification covers: every public endpoint of the Governance API across the seven services (Constitution, Traceability, Certification, SLO, Self-Audit, Release, Webhooks); every event type emitted by the governance system; the webhook subscription and delivery contract; the OAuth 2.0 authentication scheme and the RBAC authorization model; the rate-limiting, pagination, versioning, and idempotency contracts; and the Python, TypeScript, and Go SDKs and the CLI.

Out of scope. This specification does NOT cover: the internal implementation of the MRC, ATG, CCE, dashboard, or SAP (those are governed by the RC2 Roadmap); the on-disk schema of the underlying stores (Neo4j, PostgreSQL, object storage); the operations and SRE of the governance system itself (governed by the Playbook); the editorial workflows of the Opus Publica platform (governed by Volume I of the Constitution); or the scholarly-publishing integrations (Crossref, ORCID, CLOCKSS — those are governed by their own API specifications, cross-referenced from the Traceability Matrix).

1.4 Document Conventions

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119. HTTP method names (GET, POST, PUT, PATCH, DELETE) are written in uppercase as required by RFC 9110. Status codes are quoted as integers and are normative. URI paths are quoted verbatim and SHALL be matched case-sensitively.

Convention	Meaning
RFC 2119 vocabulary	MUST/SHALL/SHOULD/MAY have their RFC 2119 meanings; violations are contract violations.
HTTP method	Case-sensitive; uppercase per RFC 9110.
Status code	Normative integer per RFC 9110 §15; client MUST handle every documented status for each endpoint.
Error format	RFC 7807 Problem Details JSON; the response body of

	any 4xx or 5xx SHALL be a Problem Details object.
ID format	All governance IDs are opaque strings; clients MUST NOT parse them. Stable IDs (e.g., provision IDs) are prefixed by domain (INV-, RULE-, CRT-, REL-, EVT-).
Pagination	Cursor-based; the response includes a ``next_cursor`` field and the client requests the next page via the ``cursor`` query parameter.
Timestamps	RFC 3339 UTC with millisecond precision (e.g., 2026-08-09T14:23:11.042Z).
Content type	Request and response bodies are ``application/json`` unless the endpoint specifies otherwise (e.g., evidence upload uses ``multipart/form-data``).

1.5 References

OpenAPI 3.1. OpenAPI Initiative. OpenAPI Specification 3.1.0. <https://swagger.io/specification/>

RFC 9110. Fielding, R., Reschke, J. HTTP Semantics, June 2022. <https://www.rfc-editor.org/rfc/rfc9110>

RFC 7807. Nottingham, M., Bryan, P. Problem Details for HTTP APIs, March 2016. <https://www.rfc-editor.org/rfc/rfc7807>

RFC 8259. Bray, T. The JavaScript Object Notation (JSON) Data Interchange Format, December 2017. <https://www.rfc-editor.org/rfc/rfc8259>

RFC 6749. Hardt, D. The OAuth 2.0 Authorization Framework, October 2012. <https://www.rfc-editor.org/rfc/rfc6749>

RFC 7519. Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT), May 2015. <https://www.rfc-editor.org/rfc/rfc7519>

RFC 2119. Bradner, S. Key words for use in RFCs to Indicate Requirement Levels, March 1997. <https://www.rfc-editor.org/rfc/rfc2119>

RFC 3339. Klyne, G., Newman, C. Date and Time on the Internet: Timestamps, July 2002. <https://www.rfc-editor.org/rfc/rfc3339>

CloudEvents 1.0. CNCF CloudEvents Specification 1.0.2. <https://cloudevents.io/>

JSON Schema 2020-12. JSON Schema Specification, Draft 2020-12. <https://json-schema.org/draft/2020-12/schema>

OSCAL. NIST OSCAL — Open Security Controls Assessment Language. <https://pages.nist.gov/OSCAL/>

Chapter 2 — API Overview

2.1 Architecture

The Governance API is the read-write surface of the RC2 governance system. It is backed by five pillars whose internal architecture is specified in the RC2 Evolution Roadmap: the Machine-Readable Constitution (MRC), the Automated Traceability Graph (ATG) backed by a Neo4j property graph, the Continuous Certification Engine (CCE) backed by an OPA/Rego policy engine, the Governance Dashboard backed by Grafana and Backstage, and the Self-Auditing Platform (SAP) backed by an anomaly-detection pipeline. The API is the single integration seam through which these pillars are exposed to the outside world; no pillar is queried directly by an integration, and no pillar exposes its own ad-hoc protocol.

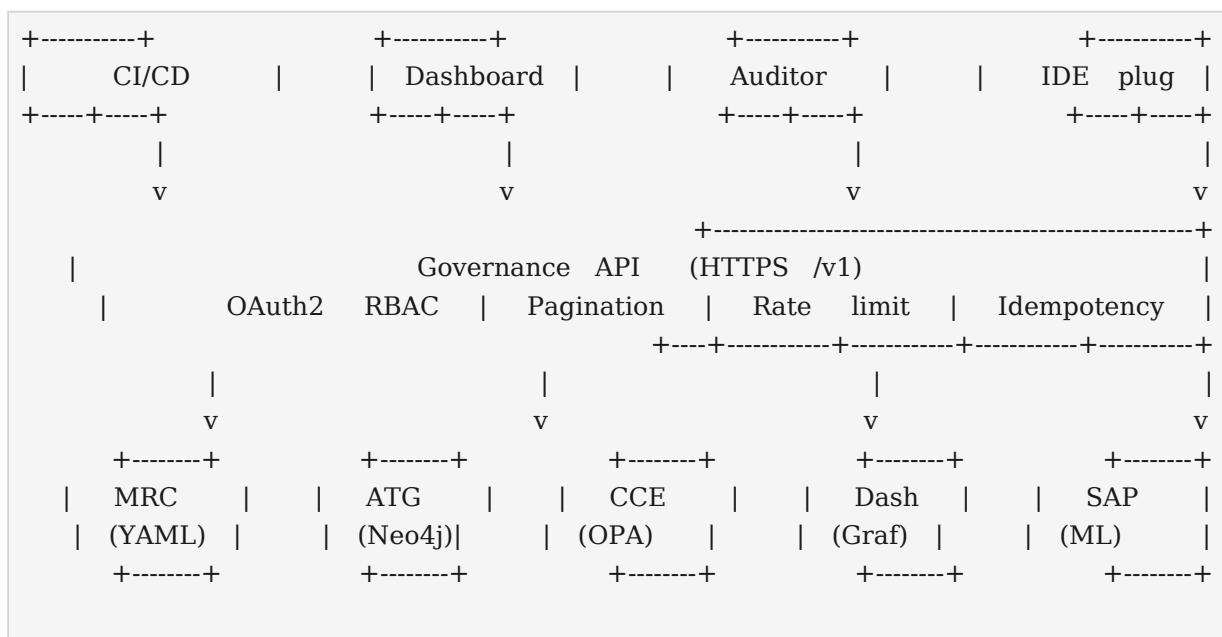


Figure 2.1 — The Governance API as the single integration seam above the five pillars.

The API itself is implemented as a stateless service tier behind a load balancer. All state lives in the pillar stores: the MRC reads from the constitution Git repository; the ATG reads and writes Neo4j; the CCE reads the OPA bundle cache and writes assessment results to PostgreSQL; the dashboard service reads Grafana snapshots; the SAP reads the anomaly-detection store. Because the API tier is stateless, it horizontally scales to thousands of concurrent connections, and a single integration may pin to any API instance without session affinity.

2.2 Base URLs and Environments

The Governance API is deployed to three environments. The base URL contains the API major version, following the URL path versioning convention (see §2.8). All paths in this specification are relative to the base URL.

Environment	Base URL	Purpose	SLA
-------------	----------	---------	-----

Development	https://gov-api.dev.opuspublica.com/v1	Engineer integration; nightly reset of seed data; no SLA.	Best effort
Staging	https://gov-api.staging.opuspublica.com/v1	Pre-production verification; weekly reconciliation with prod data shape; 99.5% monthly uptime.	99.5%
Production	https://gov-api.opuspublica.com/v1	Live governance data; consumed by production CI, dashboards, and auditors.	99.9% monthly uptime

Integrations SHALL be parameterised by environment. Hard-coding any single base URL is a defect. The SDKs accept the environment as a constructor argument; the CLI accepts it via the ``--env`` flag or the ``OP\_GOV\_ENV`` environment variable.

2.3 Authentication

The Governance API uses OAuth 2.0 client credentials grant (RFC 6749 §4.4) as the sole authentication mechanism. Integrations are identified by a service account with a pair of credentials (``client\_id``, ``client\_secret``) issued by the governance administrator. The service account presents these credentials to the token endpoint and receives a bearer access token in the form of a JWT (RFC 7519). The token is then presented in the ``Authorization: Bearer <token>`` header on every subsequent request.

```

POST                                /oauth/token                                HTTP/1.1
Host:                                auth.opuspublica.com
Content-Type:                        application/x-www-form-urlencoded

grant_type=client_credentials&
client_id=ci-orchestrator@opuspublica&
client_secret=<secret>&
scope=constitution:read              traceability:read              certification:evaluate

```

The token endpoint returns a JSON object containing the access token, its expiry, and the granted scopes. Tokens are short-lived (one hour maximum). SDKs SHALL refresh tokens proactively before expiry. Service account credentials SHALL be stored in a secret manager (Vault, AWS Secrets Manager, GitHub Actions secrets), never in source control.

2.4 Authorization

Authorization is enforced by Role-Based Access Control (RBAC). Each service account is bound to one or more roles; each role grants a set of scopes; each scope authorizes a specific verb on a specific resource family. Roles are administrative; scopes are enforced at the API gateway on every request. The five canonical roles are:

Role	Typical Bearer	Scopes Granted
reader	Dashboard service, external auditor (read-only).	constitution:read, traceability:read, certification:read, slo:read, audit:read, release:read
operator	CI/CD orchestrator; on-call engineer.	all reader scopes, plus traceability:write (evidence upload), audit:write (remediation), release:deploy.
certifier	Release Certification Authority (RCA).	all operator scopes, plus certification:evaluate, certification:signoff.
auditor	External regulator; internal audit.	constitution:read, traceability:read, certification:read, audit:read, plus audit:export (bulk evidence export).
admin	Governance administrator (CSA office).	all scopes, plus constitution:write (amendments), webhook:admin, account:admin.

2.5 Rate Limiting

The Governance API enforces per-scope rate limits to protect the pillar stores from excessive load. Limits are per service account and per scope, not per IP. When a limit is exceeded, the API responds with HTTP 429 (Too Many Requests) and includes a ``Retry-After`` header indicating the number of seconds the client MUST wait before retrying. The 429 response body is an RFC 7807 Problem Details object whose ``type`` is ``https://gov-api.opuspublica.com/errors/rate-limited``.

Scope	Limit (per minute)	Burst
constitution:read	600	1200
traceability:read	600	1200
traceability:write	60	120
certification:evaluate	30	60
audit:read	300	600

release:deploy	10	20
----------------	----	----

Clients SHALL implement exponential backoff with jitter on 429 responses. The SDKs implement this automatically; raw HTTP clients MUST implement it explicitly. Repeated 429s without backoff MAY result in temporary account suspension.

2.6 Pagination

List endpoints SHALL use cursor-based pagination. The response body of a list endpoint is a JSON object containing a `data` array, a `next_cursor` string (or `null` if no more results), and a `has_more` boolean. The client requests the next page by passing `next_cursor` as the `cursor` query parameter on the subsequent request. Clients MUST NOT assume that cursors are stable across edits to underlying data; a cursor is valid for at most 24 hours after issuance.

GET	/v1/constitution?class=INVARIANT&severity=SEV-1&page_size=50	HTTP/1.1
Host:		gov-api.opuspublica.com
Authorization:	Bearer	<token>
HTTP/1.1	200	OK
Content-Type:		application/json
<pre>{ "data": [{ "id": "INV-I-01", "next_cursor": "eyJzZXEiOjUwfQ==", "has_more": true }, ...], ... }</pre>		

The `page_size` query parameter is honoured up to a server-side maximum of 200; values above 200 are silently capped. The default page size is 50. The server SHALL return `400 Bad Request` if `page_size` is negative or non-numeric. Offset-based pagination is NOT supported; it is too expensive on the underlying Neo4j graph for deep pages.

2.7 Error Handling

All error responses (4xx and 5xx) SHALL use the RFC 7807 Problem Details JSON format. The response Content-Type SHALL be `application/problem+json`. A Problem Details object has the following fields:

Field	Type	Description
type	string (URI)	A URI reference identifying the problem type. Defaults to <code>about:blank</code> if no specific type.
title	string	A short human-readable summary of the problem type.

status	integer	The HTTP status code generated by the origin server for this occurrence.
detail	string	A human-readable explanation specific to this occurrence.
instance	string (URI)	A URI reference identifying this specific occurrence; useful for support correlation.
trace_id	string	W3C Trace Context identifier; cross-references OpenTelemetry spans for server-side diagnostics.
errors	array (optional)	For 422 validation errors: an array of field-level validation errors with ``field`` and ``message``.

```

HTTP/1.1          422          Unprocessable          Entity
Content-Type:      application/problem+json
{
  "type":          "https://gov-api.opuspublica.com/errors/validation",
  "title":         "Validation failed",
  "status":        422,
  "detail":        "The request body failed schema validation.",
  "instance":      "/v1/traceability/evidence",
  "trace_id":      "4f5e6d7c8b9a0f1e",
  "errors":        [
    { "field": "artifact_uri", "message": "must be a valid URI" },
    { "field": "evidence_type", "message": "must be one of [scan, log, screenshot, query,
reconcil]"
  }
]
}

```

The full error code catalog is reproduced in Appendix B. Every error type URI in the catalog resolves to a human-readable page explaining the error and the recommended remediation. Clients SHALL treat unknown ``type`` URIs as the generic error of the same status code.

2.8 Versioning

The Governance API uses URL path versioning: every endpoint URI begins with ``/v1/``. Major version increments (``/v2/``) introduce breaking changes and SHALL be released alongside the previous major version for a deprecation window of at least twelve months. Within a major version, the API follows Semantic Versioning semantics: additive changes (new endpoints, new optional fields, new enum values) increment the minor version and are non-breaking; bug fixes increment the patch version. The full API version is reported in the ``X-API-Version`` response header (e.g., ``1.4.2``).

Deprecation. An endpoint or field marked deprecated SHALL emit a ``Deprecation`` response header (RFC 9745) and a ``Sunset`` header naming the removal date. Clients SHOULD alert on the presence of these headers. A deprecated endpoint SHALL remain available until the Sunset date; after Sunset, the endpoint returns ``410 Gone``.

2.9 Idempotency

All POST and PUT endpoints accept an ``Idempotency-Key`` header. The key is a client-generated UUID (RFC 4122) that uniquely identifies the operation. The server stores the key and the response for 48 hours; if a request with the same key is received within the window, the server replays the original response without re-executing the operation. This contract enables safe retries from CI pipelines and from SDKs with automatic retry.

The idempotency record is scoped to the service account; the same key submitted by a different account is treated as a different operation. If the same key is submitted with a different request body, the server responds ``422 Unprocessable Entity`` with the ``idempotency-key-reuse`` error type. Idempotency keys are case-sensitive and SHALL be treated as opaque by the client.

POST	/v1/traceability/evidence	HTTP/1.1
Host:		gov-api.opuspublica.com
Authorization:	Bearer	<token>
Idempotency-Key:		8f14e45f-ceed-467f-a830-a8706966b84f
Content-Type:		application/json
{	"evidence_type": "scan",	"artifact_uri": "s3://..." }

Chapter 3 — The OpenAPI 3.1 Specification

This chapter presents the structure of the OpenAPI 3.1 document that describes the Governance API. The full document is reproduced in Appendix A; this chapter unpacks the structure so that integration authors and SDK authors can navigate it efficiently. OpenAPI 3.1 is the version targeted because it aligns the OpenAPI Schema Object with JSON Schema Draft 2020-12, allowing the governance system to reuse the same schema definitions for runtime validation and for client generation.

3.1 Top-Level Structure

The OpenAPI document has the seven top-level fields required of any 3.1 specification: ``openapi``, ``info``, ``servers``, ``tags``, ``security``, ``paths``, and ``components``. The ``webhooks`` field is also populated to declare the events the API emits (see Chapter 10). The ``jsonSchemaDialect`` is set to the JSON Schema Draft 2020-12 dialect URI to make the schema semantics explicit.

```

openapi: 3.1.0
jsonSchemaDialect: https://json-schema.org/draft/2020-12/schema
info:
  title: Opus Publica Governance API
  version: 1.0.0
  description: |
    The read-write surface of the RC2 governance system. Every governance primitive
    is addressable by a stable URI; every mutation is authenticated, authorized,
    idempotent, and immutable.
  contact:
    name: Office of the Chief Systems Architect
    email: gov-api@opuspublica.com
  license:
    name: Apache 2.0
    url: https://www.apache.org/licenses/LICENSE-2.0.html
servers:
  - url: https://gov-api.dev.opuspublica.com/v1
    description: Development
  - url: https://gov-api.staging.opuspublica.com/v1
    description: Staging
  - url: https://gov-api.opuspublica.com/v1
    description: Production
tags:
  - name: Constitution
    description: Read and validate the machine-readable constitution.
  - name: Traceability
    description: Query the traceability graph and upload evidence.
  - name: Certification
    description: Evaluate criteria and manage release sign-off.
  - name: SLO
    description: Read SLO definitions and live SLI values.

```

security:	-		name:		Audit	
	description:	Read	nightly	audit	findings	and anomalies.
	-		name:		Release	
	description:	Authorize	and	inspect	releases.	
	-		name:		Webhooks	
	description:	Subscribe	to	and	verify	webhook deliveries.
	-		OAuth2:		[]	

3.2 The components/schemas (Data Models)

The schema catalogue defines the canonical data models that flow through the API. Every schema is a JSON Schema 2020-12 object with ``\$id`` set to a stable URI so that schemas can be referenced from external validators. The schemas fall into seven families, mirroring the seven services.

Schema Family	Representative Schemas	Purpose
Constitution	Provision, ProvisionVersion, ConstitutionDiff	Represents constitutional provisions and their version history.
Traceability	TraceabilityNode, TraceabilityEdge, TraceabilityGraph, EvidenceArtifact, IntegrityReport	Represents nodes and edges in the 13-node traceability chain and the evidence attached to each edge.
Certification	Criterion, AssessmentResult, CertificationRecord, SignOff	Represents certification criteria and their evaluation results.
SLO	SLO, SLIValue, BurnRatePoint, ErrorBudget	Represents SLO definitions and live measurements.
Audit	NightlyReport, Finding, Anomaly, Remediation	Represents outputs of the self-auditing platform.
Release	ReleaseRecord, AcceptanceGate, DeploymentAuthorization	Represents release artifacts and the production acceptance gate state.
Common	Problem, Page, Cursor, CloudEvent, WebhookSubscription, WebhookDelivery	Shared schemas used across all services.

Each schema is reproduced here in summary form. The full definitions are in Appendix A. The Provision schema is the central schema of the Constitution family.

Provision:									
				type:				object	
required:	[id,	class,	severity,	status,	title,	body,	owner,	version]	
								properties:	
								id:	

```

    type: string
    pattern: '^(INV|RULE|CONTRACT|FIELD|EVT|SLO|THR|SM|SEC|CRT|GATE)-[A-Z0-9-]+$'
    description: Stable constitutional ID of the provision.
    class:
      type: string
    enum: [INVARIANT, RULE, CONTRACT, FIELD, EVENT, SLO, THRESHOLD,
          STATE_MACHINE, SECURITY, CRITERION, GATE]
    severity:
      type: string
      enum: [SEV-1, SEV-2, SEV-3]
    status:
      type: string
      enum: [ACTIVE, DEPRECATED, SUPERSEDED, DRAFT]
    title:
      type: string
      maxLength: 200
    body:
      type: string
    description: The normative text of the provision (Markdown).
    owner:
      type: string
    description: The role accountable for the provision.
    version:
      type: string
      pattern: '^\\d+\\.\\d+\\.\\d+$'
    source:
      type: object
      properties:
        document: { type: string }
        chapter: { type: string }
        section: { type: string }
    traceability_root:
      type: string
    description: URI of the root node in the traceability graph for this provision.
    additionalProperties: false

```

The Criterion schema is the central schema of the Certification family. A Criterion is the machine-readable form of a checklist entry from the RC1 Certification Checklist; each criterion has a Rego policy attached that the CCE evaluates.

```

Criterion:
  type: object
  required: [id, provision_id, gating, pass_condition, rego_rule_uri]
  properties:
    id:

```

```

      type: string
      pattern: '^CRT-[A-Z0-9-]+$'
      provision_id:
        type: string
description: The provision this criterion certifies.
      gating:
        type: string
      enum: [GATE, INFO]
      pass_condition:
        type: string
description: The objective, binary condition that MUST hold to PASS.
      fail_condition:
        type: string
      verification_method:
        type: string
      enum: [automated, query, visual, reconciliation]
      max_evidence_age_seconds:
        type: integer
        minimum: 60
      rego_rule_uri:
        type: string
        format: uri
description: URI of the Rego rule that evaluates this criterion.
      verifier_role:
        type: string

```

The EvidenceArtifact schema represents the immutable record of evidence that supports an assessment result. Evidence is stored in object storage; the API record carries the URI, the SHA-256 hash, and the metadata required for verification.

```

EvidenceArtifact:
      type: object
      required: [id, evidence_type, artifact_uri, sha256, collected_at, collected_by]
      properties:
        id:
          type: string
          pattern: '^EV-[A-Z0-9-]+$'
        evidence_type:
          type: string
      enum: [scan, log, screenshot, query, reconciliation, attestation]
        artifact_uri:
          type: string
          format: uri
        sha256:
          type: string
          pattern: '^[a-f0-9]{64}$'

```



```

                                collected_at:
                                  type:      string
                                  format:    date-time
                                collected_by:
                                  type:      string
description: The service account that produced the evidence.
                                expires_at:
                                  type:      string
                                  format:    date-time
description: After this instant, evidence is treated as stale (FAIL).
```

3.3 The components/securitySchemes

The OpenAPI document declares a single security scheme: an OAuth2 scheme using the client credentials flow. The token URL is shared across all environments; the environment is determined by the requested scope prefix. The flow declares the full scope catalogue; every endpoint references a subset of these scopes via the ``security`` field on the operation.

```

components:
                                securitySchemes:
                                    OAuth2:
                                        type:      oauth2
                                        flows:
                                            clientCredentials:
tokenUrl: https://auth.opuspublica.com/oauth/token
                                scopes:
constitution:read: Read provisions, versions, and diffs.
constitution:write: Amend the constitution (admin only).
traceability:read: Query the graph and read evidence.
traceability:write: Upload evidence and create nodes.
certification:read: Read criteria and assessment results.
certification:evaluate: Synchronously evaluate a criterion.
certification:signoff: Record RCA sign-off on a release.
slo:read: Read SLO definitions and live SLI values.
audit:read: Read findings and anomalies.
audit:write: Record remediations.
audit:export: Bulk-export evidence for external audit.
release:read: Read release records and gate state.
release:deploy: Authorize a deployment (operator only).
webhook:admin: Subscribe, list, and revoke webhooks.
account:admin: Manage service accounts (admin only).
```

3.4 The components/responses

The OpenAPI document declares four reusable response objects for the standard error cases. Every operation in the document references one or more of these via ``\$ref``, ensuring that the error contract is uniform across endpoints. The four reusable responses are: ``Unauthorized`` (401), ``Forbidden`` (403), ``NotFound`` (404), and ``Validation`` (422). A fifth reusable response, ``RateLimited`` (429), is also declared.

```
components:
  responses:
    Unauthorized:
      description: The request lacks valid authentication credentials.
      content:
        application/problem+json:
          schema: { $ref: '#/components/schemas/Problem' }
    Forbidden:
      description: The authenticated principal lacks the required scope.
      content:
        application/problem+json:
          schema: { $ref: '#/components/schemas/Problem' }
    NotFound:
      description: The requested resource does not exist.
      content:
        application/problem+json:
          schema: { $ref: '#/components/schemas/Problem' }
    Validation:
      description: The request body failed schema validation.
      content:
        application/problem+json:
          schema: { $ref: '#/components/schemas/Problem' }
    RateLimited:
      description: The caller exceeded the per-scope rate limit.
      headers:
        Retry-After:
          schema: { type: integer, minimum: 1 }
      content:
        application/problem+json:
          schema: { $ref: '#/components/schemas/Problem' }
```

The full OpenAPI document, with all paths, all schemas, and all reusable components, is reproduced in Appendix A. The remainder of this specification (Chapters 4 through 9) documents each endpoint in narrative form, with examples and error cases that the OpenAPI document cannot itself express.

Chapter 4 — Endpoint Reference: Constitution Service

The Constitution Service exposes the machine-readable constitution (MRC) as a read-only API. The MRC is the canonical form of the RC1 Engineering Constitution, encoded as YAML in a Git repository; this service reads the YAML and presents it as JSON over HTTPS. Amendments to the constitution are made by pull request against the repository and are not performed through this API; the API therefore exposes only one write operation — a dry-run validation of a proposed provision.

4.1 GET /v1/constitution — List Provisions

GET /v1/constitution — *List constitutional provisions, with filters.*

Description

Returns a paginated list of provisions in the active constitution. The list MAY be filtered by class, severity, owner, and status. The default sort is by provision ID ascending. This endpoint is the primary read path for the Constitution family and is consumed by the dashboard, by the CI orchestrator (to determine which criteria apply to a release), and by external auditors.

Parameters

Name	In	Required	Type	Description
class	query	no	string (enum)	Filter by provision class. One of INVARIANT, RULE, CONTRACT, FIELD, EVENT, SLO, THRESHOLD, STATE_MACHINE, SECURITY, CRITERION, GATE.
severity	query	no	string (enum)	Filter by severity. One of SEV-1, SEV-2, SEV-3.
owner	query	no	string	Filter by owner role. The value is matched case-sensitively.
status	query	no	string (enum)	Filter by status. One of ACTIVE, DEPRECATED, SUPERSEDED, DRAFT. Defaults to ACTIVE.
cursor	query	no	string	Opaque pagination cursor returned by a

				previous call.
page_size	query	no	integer	Page size; default 50, maximum 200.

Responses

Status	Description
200	A paginated list of Provision objects.
400	Invalid query parameter (Problem Details).
401	Missing or invalid bearer token (Problem Details).
403	Token lacks the constitution:read scope (Problem Details).
429	Rate limit exceeded; see Retry-After header.

Security

Requires the ``constitution:read`` scope. The ``reader``, ``operator``, ``certifier``, ``auditor``, and ``admin`` roles all grant this scope.

Example Request

```
GET /v1/constitution?class=INVARIANT&severity=SEV-1&page_size=10 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Accept: application/json
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
X-API-Version: 1.0.0
{
  "data": [
    {
      "id": "INV-I-01",
      "class": "INVARIANT",
      "severity": "SEV-1",
      "status": "ACTIVE",
      "title": "Exactly one canonical article",
      "body": "For every published Work W, COUNT(canonical_articles WHERE work_id = W) MUST equal 1.",
      "owner": "RCA",
      "version": "1.0.0",
      "source": { "document": "Vol II Ch 26", "chapter": "26", "section": "26.1" },
    }
  ]
}
```

```

      "traceability_root":    "/v1/traceability/requirements/INV-I-01/chain"
                                }
                                ],
      "next_cursor":          "eyJzZXEiOjEwfQ==",
      "has_more":             true
    }

```

Error Cases

- **constitution:invalid-filter** — Returned when an enum filter value is not in the allowed set.
- **pagination:bad-cursor** — Returned when the cursor is malformed or expired.

4.2 GET /v1/constitution/{provision_id} — Get a Single Provision

GET /v1/constitution/{provision_id} — Retrieve a single provision by its stable ID.

Description

Returns the full Provision object identified by ``provision\_id``, including its normative body, source citation, owner, version, and the URI of the root of its traceability chain. This is the canonical read path for a single provision and is the URL that dashboards and IDE plugins link to when surfacing a provision.

Parameters

Name	In	Required	Type	Description
provision_id	path	yes	string	The stable constitutional ID of the provision (e.g., ``INV-I-01``).

Responses

Status	Description
200	The Provision object.
401	Missing or invalid bearer token.
403	Token lacks the constitution:read scope.
404	No provision with the given ID exists (or it has been removed).
429	Rate limit exceeded; see Retry-After header.

Security

Requires the ``constitution:read`` scope.

Example Request

```
GET /v1/constitution/INV-I-01 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Accept: application/json
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": "INV-I-01",
  "class": "INVARIANT",
  "severity": "SEV-1",
  "status": "ACTIVE",
  "title": "Exactly one canonical article",
  "body": "For every published Work W, COUNT(canonical_articles WHERE work_id = W) MUST equal 1.",
  "owner": "RCA",
  "version": "1.0.0",
  "source": {
    "document": "Vol II Ch 26",
    "chapter": "26",
    "section": "26.1"
  },
  "traceability_root": "/v1/traceability/requirements/INV-I-01/chain"
}
```

Error Cases

- **constitution:not-found** — The provision ID does not match any active or historical provision.
- **constitution:malformed-id** — The provision ID does not match the documented prefix pattern.

4.3 GET /v1/constitution/{provision_id}/traceability — Get the Traceability Chain

GET /v1/constitution/{provision_id}/traceability — Get the 13-node traceability chain rooted at this provision.

Description

Returns the full 13-node traceability chain for the given provision, from the provision itself through architecture decisions, code modules, tests, runtime evidence, certification results, and the audit record. The response is a `TraceabilityGraph` object containing `nodes` and `edges`. This endpoint is a convenience wrapper around `GET /v1/traceability/requirements/{id}/chain`; the two endpoints return the same data.

Parameters

Name	In	Required	Type	Description
provision_id	path	yes	string	The provision ID whose chain is requested.
depth	query	no	integer	Maximum graph depth; default 13 (the full canonical chain), maximum 13.

Responses

Status	Description
200	A TraceabilityGraph object containing the chain.
401	Missing or invalid bearer token.
403	Token lacks the constitution:read scope.
404	Provision not found, or has no traceability chain.
429	Rate limit exceeded.

Security

Requires ``constitution:read`` (the traceability data is read through this scope).

Example Request

```
GET /v1/constitution/INV-I-01/traceability HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "root": "INV-I-01",
  "nodes": [
    { "id": "INV-I-01", "type": "PROVISION", "label": "Exactly one canonical article" },
    { "id": "ADR-0042", "type": "ARCHITECTURE_DECISION", "label": "Canonical-article store" },
    { "id": "MOD-canonical", "type": "MODULE", "label": "canonical_article_service" },
    { "id": "T-canonical-01", "type": "TEST", "label": "test_canonical_article_count" }
  ]
}
```

```

      "edges": [
        { "from": "INV-I-01", "to": "ADR-0042", "kind": "REALIZED_BY" },
        { "from": "ADR-0042", "to": "MOD-canonical", "kind": "IMPLEMENTED_IN" },
        { "from": "MOD-canonical", "to": "T-canonical-01", "kind": "VERIFIED_BY" }
      ],
      "integrity": { "passed": 7, "failed": 0, "rules_run": ["IR-01","IR-02","IR-03"] }
    }
  }

```

Error Cases

- **traceability:no-chain** — The provision exists but no traceability chain has been built.
- **traceability:depth-exceeded** — The requested depth exceeds the maximum of 13.

4.4 POST /v1/constitution/validate — Validate a Provision (Dry-Run)

/v1/constitution/validate — *Validate a proposed provision YAML without persisting it.*

Description

Accepts a proposed provision in YAML form and runs the same JSON Schema and Rego validation that the constitution amendment pipeline runs, without writing the provision to the constitution. Returns a list of validation errors, or an empty list if the provision is valid. CI pipelines and IDE plugins use this endpoint to give authors fast feedback on proposed amendments before they open a pull request. The endpoint is idempotent: the same input always produces the same output, and the ``Idempotency-Key`` header is honoured but optional.

Request Body

The request body is a JSON object with a single field, ``yaml``, containing the proposed provision as a YAML string. The maximum body size is 64 KiB.

```

POST /v1/constitution/validate HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Idempotency-Key: 8f14e45f-ceeda-467f-a830-a8706966b84f
Content-Type: application/json
{
  "yaml": "id: INV-I-42\nclass: INVARIANT\nseverity: SEV-1\ntitle: ...\\n"
}

```

Responses

Status	Description
200	Validation completed; the response contains any errors found.
400	The request body was not valid JSON or exceeded the size

	limit.
401	Missing or invalid bearer token.
403	Token lacks the constitution:read scope.
413	Request body too large.
422	Validation could not be performed (e.g., YAML parse error).
429	Rate limit exceeded.

Security

Requires the ``constitution:read`` scope. The validate endpoint is open to all readers because it performs no mutation.

Example Request

```
POST /v1/constitution/validate HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Content-Type: application/json
{
  "yaml": "id: INV-I-42\nclass: INVARIANT\nseverity: SEV-1\ntitle: Example\nbody: ...
\nowner: RCA\n"
}
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "valid": false,
  "errors": [
    { "path": "/body", "message": "Property is required." },
    { "path": "/version", "message": "Does not match pattern ^\\d+\\.\\d+\\.\\d+$." }
  ]
}
```

Error Cases

- **constitution:yaml-parse-error** — The submitted YAML could not be parsed.
- **constitution:schema-error** — The submitted YAML parsed but did not match the Provision schema.

4.5 GET /v1/constitution/versions — List Constitution Versions

GET /v1/constitution/versions — List all published versions of the constitution.

Description

Returns a paginated list of every published version of the constitution, ordered by publication timestamp descending. Each entry contains the SemVer tag, the publication timestamp, the publishing CSA, the Git commit SHA, and a summary of the amendments included in that version. This endpoint is the audit trail of constitutional change.

Parameters

Name	In	Required	Type	Description
cursor	query	no	string	Pagination cursor.
page_size	query	no	integer	Page size; default 50, max 200.
since	query	no	string (date-time)	Return only versions published at or after this instant.
until	query	no	string (date-time)	Return only versions published before this instant.

Responses

Status	Description
200	Paginated list of ConstitutionVersion objects.
400	Invalid query parameter.
401	Missing or invalid bearer token.
403	Token lacks the constitution:read scope.
429	Rate limit exceeded.

Security

Requires the ``constitution:read`` scope.

Example Request

```
GET /v1/constitution/versions?page_size=5 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```

HTTP/1.1                200                OK
Content-Type:            application/json
{
    "data": [
        {
            "version": "1.4.0",
            "published_at": "2026-08-09T14:23:11.042Z",
            "published_by": "csa@opuspublica.com",
            "git_sha": "a1b2c3d4e5f6...",
            "summary": "Amend INV-I-01 to require hourly reconciliation."
        }
    ],
    "next_cursor": null,
    "has_more": false
}

```

4.6 GET /v1/constitution/diff — Get a Constitutional Diff

GET /v1/constitution/diff — Compute the structural diff between two constitution versions.

Description

Returns a ConstitutionDiff object describing every change between two versions of the constitution: provisions added, provisions removed, provisions amended (with field-level diffs), and severity changes. The diff is computed using RFC 6902 JSON Patch operations, expressed as a list of op/path/value triples. This endpoint is the machine-readable form of the human-readable constitutional changelog.

Parameters

Name	In	Required	Type	Description
from	query	yes	string	The SemVer tag of the source version.
to	query	yes	string	The SemVer tag of the target version.
format	query	no	string (enum)	One of ``summary`` (default) or ``patch`` (RFC 6902 operations).

Responses

Status	Description
200	A ConstitutionDiff object.

400	Missing ``from`` or ``to`` parameter.
401	Missing or invalid bearer token.
403	Token lacks the constitution:read scope.
404	One of the named versions does not exist.
422	The SemVer strings are malformed.
429	Rate limit exceeded.

Security

Requires the ``constitution:read`` scope.

Example Request

```
GET /v1/constitution/diff?from=1.3.0&to=1.4.0 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "from": "1.3.0",
  "to": "1.4.0",
  "summary": {
    "added": [],
    "removed": [],
    "amended": [ "INV-I-01" ],
    "severity_changed": []
  },
  "patch": [
    { "op": "replace", "path": "/INV-I-01/body", "value": "...new body..." },
    { "op": "replace", "path": "/INV-I-01/version", "value": "1.4.0" }
  ]
}
```

Error Cases

- **constitution:version-not-found** — One of the ``from`` or ``to`` versions does not exist.
- **constitution:diff-too-large** — The diff exceeds 500 operations; use the per-provision endpoints instead.

Chapter 5 — Endpoint Reference: Traceability Service

The Traceability Service exposes the Automated Traceability Graph (ATG), a Neo4j property graph containing the 13-node chain that links every constitutional provision to its architectural decision, code module, test, runtime evidence, certification result, and audit record. Reads are open to all readers; the only write operation is evidence upload, used by CI to record fresh evidence after a build.

5.1 GET /v1/traceability/graph — Query the Graph

GET /v1/traceability/graph — Query the traceability graph (structured query or Cypher passthrough).

Description

Returns the subgraph matching the supplied query. The endpoint accepts a structured query (a JSON object with ``start``, ``hop``, ``filter``, and ``limit`` fields) which is the recommended form for integrations. The Cypher passthrough form (``?cypher=MATCH ...``) is restricted to the ``operator`` role and above and is intended for ad-hoc investigation only; production integrations SHALL use the structured form.

Parameters

Name	In	Required	Type	Description
start	query	yes	string	The node ID at which to start the traversal.
hop	query	no	integer	Maximum hops from ``start``. Default 3, maximum 13.
filter	query	no	string (JSON)	A JSON object of node-property filters, e.g., ``{"type": ["MODULE", "TEST"]}``.
limit	query	no	integer	Maximum number of nodes returned. Default 100, maximum 1000.
cypher	query	no	string	A Cypher query (operator-only). Mutually exclusive with the structured parameters.

Responses

Status	Description
200	A TraceabilityGraph object.
400	Invalid query parameter.
401	Missing or invalid bearer token.
403	Token lacks the traceability:read scope (or, for ``cypher``, lacks the operator role).
422	The structured filter is not valid JSON, or the Cypher query is syntactically invalid.
429	Rate limit exceeded.

Security

Requires ``traceability:read``. The ``cypher`` parameter additionally requires the ``operator`` role.

Example Request

```
GET /v1/traceability/graph?start=INV-I-01&hop=3&limit=50 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "root": "INV-I-01",
  "nodes": [ { "id": "INV-I-01", "type": "PROVISION", "label": "..."}, ... ],
  "edges": [ { "from": "INV-I-01", "to": "ADR-0042", "kind": "REALIZED_BY"}, ... ],
  "truncated": false
}
```

Error Cases

- **traceability:no-such-node** — The ``start`` node does not exist.
- **traceability:cypher-rejected** — The Cypher query was rejected by the safety filter (e.g., contains a write operation).

5.2 GET /v1/traceability/requirements/{id}/chain — Get the Full 13-Node Chain

GET /v1/traceability/requirements/{id}/chain — *Get the full canonical 13-node traceability chain for a requirement.*

Description

Returns the complete 13-node chain for the given requirement ID, from the requirement through to the audit record. This endpoint is the canonical read path for a single requirement's chain; it is more efficient than ``GET /v1/traceability/graph`` for this specific shape of query because it uses a precomputed traversal.

Parameters

Name	In	Required	Type	Description
id	path	yes	string	The requirement ID (a provision ID or a derived requirement ID).
include_evidence	query	no	boolean	If true, include EvidenceArtifact objects attached to each edge. Default false.

Responses

Status	Description
200	A TraceabilityGraph object with up to 13 nodes.
401	Missing or invalid bearer token.
403	Token lacks the traceability:read scope.
404	No chain exists for this requirement.
429	Rate limit exceeded.

Security

Requires the ``traceability:read`` scope.

Example Request

```
GET      /v1/traceability/requirements/INV-I-01/chain?include_evidence=true  HTTP/1.1
Host:    gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "root": "INV-I-01",
  "nodes": [ /* up to 13 nodes */ ],
  "edges": [ /* up to 12 edges */ ],
  "evidence": [ { "edge": "MOD-canonical->T-canonical-01", "artifact": "EV-..." } ],
  "integrity": { "passed": 7, "failed": 0 }
}
```

Error Cases

- **traceability:incomplete-chain** — The chain exists but has fewer than 13 nodes; the response still returns 200 with the nodes it has.

5.3 GET /v1/traceability/orphans — List Orphan Nodes

GET /v1/traceability/orphans — *List traceability nodes with no incoming or outgoing edges of a given kind.*

Description

Returns nodes that have no edges of the specified kind, indicating gaps in the traceability graph. CI integrations use this endpoint to detect, for example, tests that do not verify any provision (```kind=VERIFIED_BY```) or code modules that have no test (```kind=TESTED_BY```). The endpoint is the read path behind the Traceability Gap Detected event (see Chapter 10).

Parameters

Name	In	Required	Type	Description
type	query	yes	string (enum)	Filter orphans by node type (PROVISION, MODULE, TEST, EVIDENCE, etc.).
missing_edge_kind	query	yes	string (enum)	The edge kind that is missing. One of REALIZED_BY, IMPLEMENTED_IN, VERIFIED_BY, EVIDENCED_BY, CERTIFIED_BY, AUDITED_BY.
cursor	query	no	string	Pagination cursor.
page_size	query	no	integer	Page size; default 50, max 200.

Responses

Status	Description
200	Paginated list of TraceabilityNode objects.
400	Missing required query parameter.
401	Missing or invalid bearer token.
403	Token lacks the traceability:read scope.
429	Rate limit exceeded.

Security

Requires the ``traceability:read`` scope.

Example Request

```
GET    /v1/traceability/orphans?type=TEST&missing_edge_kind=VERIFIED_BY  HTTP/1.1
Host:                                     gov-api.opuspublica.com
Authorization:                           Bearer                        <token>
```

Example Response

```
HTTP/1.1                                200                                OK
Content-Type:                            application/json
{
  "data": [
    { "id": "T-orphan-07", "type": "TEST", "label": "test_legacy_importer" },
  ],
  "next_cursor": null,
  "has_more": false
}
```

5.4 GET /v1/traceability/integrity — Run Integrity Rules

GET /v1/traceability/integrity — Run the eight traceability integrity rules (IR-01 through IR-08).

Description

Executes the eight integrity rules defined by the ATG and returns a per-rule report. The rules verify that every provision is realized, every module is tested, every test verifies a provision, every evidence is referenced, every certification result cites its evidence, every audit finding cites its certification result, and the like. The full rule catalogue is in the RC2 Roadmap Chapter 3. This endpoint is invoked nightly by the self-auditing platform.

Parameters

Name	In	Required	Type	Description
rules	query	no	string (comma-separated)	Subset of rules to run; default all (`IR-01,IR-02,IR-03,IR-04,IR-05,IR-06,IR-07,IR-08`).
scope	query	no	string	Limit the run to a subgraph rooted at the given node ID.

Responses

Status	Description
200	An IntegrityReport object with per-rule pass/fail counts and example violations.
401	Missing or invalid bearer token.
403	Token lacks the traceability:read scope.
429	Rate limit exceeded.

Security

Requires the ``traceability:read`` scope.

Example Request

```
GET /v1/traceability/integrity?rules=IR-01,IR-02 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "started_at": "2026-08-09T02:00:00.000Z",
  "finished_at": "2026-08-09T02:00:11.482Z",
  "rules": [
    { "id": "IR-01", "name": "Every provision is realized", "passed": 247, "failed": 0 },
    { "id": "IR-02", "name": "Every module is tested", "passed": 189, "failed": 3 }
  ],
  "violations": [
```

```
{ "rule": "IR-02", "node": "MOD-legacy-importer", "detail": "No VERIFIED_BY edge." }
]
```

5.5 GET /v1/traceability/evidence/{id} — Retrieve an Evidence Artifact

GET /v1/traceability/evidence/{id} — Retrieve the metadata of an evidence artifact.

Description

Returns the EvidenceArtifact object identified by ``id``, including its SHA-256 hash, collection timestamp, collector, and the URI of the artifact in object storage. The artifact bytes themselves are retrieved via a separate ``GET /v1/traceability/evidence/{id}/content`` endpoint that issues a short-lived pre-signed URL.

Parameters

Name	In	Required	Type	Description
id	path	yes	string	The evidence ID (e.g., ``EV-...``).

Responses

Status	Description
200	An EvidenceArtifact object.
401	Missing or invalid bearer token.
403	Token lacks the traceability:read scope.
404	No evidence with this ID exists.
429	Rate limit exceeded.

Security

Requires the ``traceability:read`` scope.

Example Request

```
GET /v1/traceability/evidence/EV-2026-08-09-001 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```

HTTP/1.1                200                OK
Content-Type:            application/json
{
    "id":                  "EV-2026-08-09-001",
    "evidence_type":       "scan",
    "artifact_uri":        "s3://opus-evidence/EV-2026-08-09-001.json",
    "sha256":              "9e7a3b4f...c1",
    "collected_at":       "2026-08-09T01:59:58.221Z",
    "collected_by":       "ci-orchestrator@opuspublica",
    "expires_at":          "2026-08-09T03:00:00.000Z"
}

```

Error Cases

- **evidence:expired** — The evidence exists but its ``expires\_at`` is in the past; the response still returns 200 with the metadata, but the evidence is treated as stale by the certification engine.

5.6 POST /v1/traceability/evidence — Upload an Evidence Artifact

/v1/traceability/evidence — *Upload an evidence artifact (CI use).*

Description

Accepts an evidence artifact, computes its SHA-256 hash, stores the bytes in object storage, and creates the EvidenceArtifact record in the graph. The endpoint is the primary write path for CI pipelines: every build that produces evidence (a scan, a log, a query result) uploads it via this endpoint, and the evidence is then attached to the relevant traceability edge. The endpoint is idempotent on ``Idempotency-Key``; a replay returns the original response.

Request Body

The request body is ``multipart/form-data`` with two parts: ``metadata`` (a JSON object containing ``evidence\_type``, ``provision\_id``, ``edge\_id``, and ``expires\_at``), and ``file`` (the artifact bytes). The maximum artifact size is 50 MiB.

```

POST                /v1/traceability/evidence                HTTP/1.1
Host:                gov-api.opuspublica.com
Authorization:       Bearer <token>
Idempotency-Key:     8f14e45f-ceed-467f-a830-a8706966b84f
Content-Type:        multipart/form-data;                    boundary=----boundary
-----boundary
Content-Disposition: form-data;                               name="metadata"
Content-Type:        application/json
{ "evidence_type": "scan", "provision_id": "INV-I-01", "edge_id": "MOD-canonical->T-
canonical-01",      "expires_at": "2026-08-09T03:00:00Z"      }

```

```

-----boundary
Content-Disposition:      form-data;      name="file";      filename="scan.json"
Content-Type:              application/json

<bytes>
-----boundary--

```

Responses

Status	Description
201	The evidence was stored. The response body is the new EvidenceArtifact.
400	Malformed request body or missing part.
401	Missing or invalid bearer token.
403	Token lacks the traceability:write scope.
413	Artifact exceeds 50 MiB.
422	The metadata JSON failed schema validation.
429	Rate limit exceeded.

Security

Requires the ``traceability:write`` scope (operator, certifier, admin).

Example Request

```

POST      /v1/traceability/evidence      HTTP/1.1
Host:      gov-api.opuspublica.com
Authorization:      Bearer      <token>
Idempotency-Key:      8f14e45f-ceeda-467f-a830-a8706966b84f
Content-Type:      multipart/form-data;      boundary=----boundary
(body      elided)

```

Example Response

```

HTTP/1.1      201      Created
Location:      /v1/traceability/evidence/EV-2026-08-09-002
Content-Type:      application/json
{
  "id":      "EV-2026-08-09-002",
  "evidence_type":      "scan",
  "artifact_uri":      "s3://opus-evidence/EV-2026-08-09-002.json",
  "sha256":      "9e7a3b4f...c1",

```

```
}      "collected_at":      "2026-08-09T02:01:14.000Z",  
      "collected_by":      "ci-orchestrator@opuspublica",  
      "expires_at":         "2026-08-09T03:00:00.000Z"
```

Error Cases

- **evidence:edge-not-found** — The specified ``edge\_id`` does not exist in the graph; the evidence is stored but not attached (and a 422 is returned).
- **evidence:hash-mismatch** — If the client supplies a precomputed hash and it does not match the server-computed hash, the upload is rejected.

Chapter 6 — Endpoint Reference: Certification Service

The Certification Service exposes the Continuous Certification Engine (CCE). The CCE evaluates each constitutional criterion against current runtime state using an OPA/Rego policy engine. The service exposes read paths for criteria and assessment results, a synchronous evaluate endpoint for testing, and the RCA sign-off endpoint that constitutes the Production Acceptance Gate.

6.1 GET /v1/certification/criteria — List Criteria

GET /v1/certification/criteria — *List certification criteria, with filters.*

Description

Returns a paginated list of criteria. Each criterion is the machine-readable form of a checklist entry from the RC1 Certification Checklist, with the Rego rule URI that evaluates it. Filters allow narrowing by class (the class of the provision the criterion certifies), gating (GATE or INFO), and status.

Parameters

Name	In	Required	Type	Description
class	query	no	string (enum)	Filter by provision class.
gating	query	no	string (enum)	Filter by gating (GATE or INFO).
status	query	no	string (enum)	Filter by criterion status (ACTIVE, DEPRECATED). Default ACTIVE.
cursor	query	no	string	Pagination cursor.
page_size	query	no	integer	Page size; default 50, max 200.

Responses

Status	Description
200	Paginated list of Criterion objects.
400	Invalid query parameter.
401	Missing or invalid bearer token.
403	Token lacks the certification:read scope.
429	Rate limit exceeded.

Security

Requires the ``certification:read`` scope.

Example Request

```
GET /v1/certification/criteria?class=INVARIANT&gating=GATE HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "data": [
    {
      "id": "CRT-INV-I-01-a",
      "provision_id": "INV-I-01",
      "gating": "GATE",
      "pass_condition": "COUNT(canonical_articles WHERE work_id = W) = 1 for every published W.",
      "fail_condition": "COUNT != 1 OR evidence stale.",
      "verification_method": "automated",
      "max_evidence_age_seconds": 3600,
      "rego_rule_uri": "https://git.opuspublica.com/gov/rego/CRT-INV-I-01-a.rego",
      "verifier_role": "RCA"
    }
  ],
  "next_cursor": null,
  "has_more": false
}
```

6.2 GET /v1/certification/criteria/{id} — Get a Single Criterion

GET /v1/certification/criteria/{id} — Retrieve a single criterion by ID.

Description

Returns the full Criterion object identified by ``id``.

Parameters

Name	In	Required	Type	Description
id	path	yes	string	The criterion ID (e.g., ``CRT-INV-I-01-a``).

Responses

Status	Description
200	The Criterion object.
401	Missing or invalid bearer token.
403	Token lacks the certification:read scope.
404	No criterion with this ID exists.
429	Rate limit exceeded.

Security

Requires the ``certification:read`` scope.

Example Request

GET	/v1/certification/criteria/CRT-INV-I-01-a	HTTP/1.1
Host:		gov-api.opuspublica.com
Authorization:	Bearer	<token>

Example Response

HTTP/1.1	200	OK
Content-Type:		application/json
{	"id": "CRT-INV-I-01-a", "provision_id": "INV-I-01", "gating": "GATE", ... }	}

6.3 POST /v1/certification/evaluate — Evaluate a Criterion (Synchronous)

/v1/certification/evaluate — *Synchronously evaluate a criterion against current runtime state.*

Description

Runs the Rego rule for the named criterion against the current runtime state and returns the result (PASS, FAIL, or NOT_APPLICABLE) plus the evidence examined. This endpoint is the primary integration point for CI pipelines: a build runs the criterion, uploads the resulting evidence via the Traceability Service, and records the assessment result. The endpoint is rate-limited more aggressively than reads (30 per minute, see §2.5) because each evaluation compiles and runs a Rego query.

Request Body

The request body is a JSON object containing ``criterion\_id`` and an optional ``input`` object passed to the Rego rule as its input document.

```

POST /v1/certification/evaluate HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Idempotency-Key: 9d25f46b-dafb-5789-b941-b7177a87c95a
Content-Type: application/json
{
  "input": {
    "criterion_id": "CRT-INV-I-01-a",
    "work_id": "W-12345"
  }
}

```

Responses

Status	Description
200	Evaluation completed; the response contains the result and evidence summary.
400	Malformed request body.
401	Missing or invalid bearer token.
403	Token lacks the certification:evaluate scope.
404	The named criterion does not exist.
422	The input document does not match the Rego rule's input schema.
429	Rate limit exceeded (note the lower limit on this scope).

Security

Requires the ``certification:evaluate`` scope (certifier, admin).

Example Request

```

POST /v1/certification/evaluate HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Idempotency-Key: 9d25f46b-dafb-5789-b941-b7177a87c95a
Content-Type: application/json
{
  "criterion_id": "CRT-INV-I-01-a",
  "input": {
    "work_id": "W-12345"
  }
}

```

Example Response

```

HTTP/1.1 200 OK
Content-Type: application/json
{
  "criterion_id": "CRT-INV-I-01-a",

```

```

      "result": "PASS",
      "evaluated_at": "2026-08-09T02:15:33.001Z",
      "evidence_summary": { "canonical_article_count": 1, "work_id": "W-12345" },
      "rego_decision_id": "4f5e6d7c-8b9a-0f1e-2d3c-4b5a-6c7d-8e9f",
      "duration_ms": 41
    }

```

Error Cases

- **evaluation:timeout** — The Rego rule did not complete within 5 seconds; the result is indeterminate.
- **evaluation:rego-error** — The Rego rule raised an error (syntax, undefined function, etc.).

6.4 GET /v1/certification/results — List Assessment Results

GET /v1/certification/results — *List assessment results in OSCAL Assessment Results format.*

Description

Returns a paginated list of AssessmentResult objects. Each result is the machine-readable record of one criterion evaluation, including the criterion ID, the result, the evidence examined, the evaluator, and the timestamp. The endpoint also supports an ``accept=application/oscal+json`` content negotiation that returns the results as an OSCAL Assessment Results (OSCAL-AR) document, suitable for direct submission to a regulator.

Parameters

Name	In	Required	Type	Description
criterion_id	query	no	string	Filter by criterion ID.
provision_id	query	no	string	Filter by provision ID.
result	query	no	string (enum)	Filter by result (PASS, FAIL, NOT_APPLICABLE).
release_id	query	no	string	Filter by release ID.
since	query	no	string (date-time)	Return only results evaluated at or after this instant.
cursor	query	no	string	Pagination cursor.
page_size	query	no	integer	Page size; default 50, max 200.

Responses

Status	Description
--------	-------------

200	Paginated list of AssessmentResult objects, or an OSCAL-AR document if the ``Accept`` header negotiates that format.
400	Invalid query parameter.
401	Missing or invalid bearer token.
403	Token lacks the certification:read scope.
406	The ``Accept`` header requested a format the server does not produce.
429	Rate limit exceeded.

Security

Requires the ``certification:read`` scope.

Example Request

```
GET /v1/certification/results?release_id=REL-2026-08-09-001&result=FAIL HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Accept: application/json
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "data": [
    {
      "id": "AR-2026-08-09-001",
      "criterion_id": "CRT-INV-I-01-a",
      "release_id": "REL-2026-08-09-001",
      "result": "FAIL",
      "evaluated_at": "2026-08-09T02:15:33.001Z",
      "evaluated_by": "ci-orchestrator@opuspublica",
      "evidence": [ "EV-2026-08-09-002" ],
      "rego_decision_id": "4f5e6d7c-8b9a-0f1e-2d3c-4b5a-6c7d-8e9f"
    }
  ],
  "next_cursor": null,
  "has_more": false
}
```

6.5 GET /v1/certification/releases/{id}/record — Get the Certification Record

GET /v1/certification/releases/{id}/record — Get the complete certification record for a release.

Description

Returns the CertificationRecord for the named release: every criterion that was evaluated, the result of each, the evidence cited, the RCA sign-off (if any), and the final gate decision. This is the document the RCA reviews at the Production Acceptance Gate and the document an external auditor samples from.

Parameters

Name	In	Required	Type	Description
id	path	yes	string	The release ID (e.g., ``REL-2026-08-09-001``).

Responses

Status	Description
200	A CertificationRecord object.
401	Missing or invalid bearer token.
403	Token lacks the certification:read scope.
404	No release with this ID exists.
429	Rate limit exceeded.

Security

Requires the ``certification:read`` scope.

Example Request

```
GET /v1/certification/releases/REL-2026-08-09-001/record HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "release_id": "REL-2026-08-09-001",
  "criteria_total": 612,
```

```

        "criteria_pass": 611,
        "criteria_fail": 1,
        "criteria_na": 0,
        "gate": "BLOCKED",
        "signoffs": [],
        "results_uri": "/v1/certification/results?release_id=REL-2026-08-09-001"
    }

```

6.6 POST /v1/certification/releases/{id}/signoff — RCA Sign-Off

/v1/certification/releases/{id}/signoff — Record the Release Certification

Authority's sign-off on a release.

Description

Records the RCA's sign-off on the Production Acceptance Gate for the named release. Sign-off is permitted only when every GATE criterion PASSES (or is N/A with written justification). On success, the release is unblocked for deployment. The endpoint requires the ``certification:signoff`` scope, which is granted only to the ``certifier`` role, and the sign-off is attributed to the authenticated service account. Sign-off is idempotent: a second call with the same release ID and a different decision is rejected (``422``); to amend a sign-off, the CSA must first revoke it via the admin endpoint.

Request Body

The request body is a JSON object containing the decision, a justification, and the list of N/A justifications (if any).

```

POST      /v1/certification/releases/REL-2026-08-09-001/signoff      HTTP/1.1
Host:      gov-api.opuspublica.com
Authorization: Bearer <token>
Idempotency-Key: 1a2b3c4d-5e6f-7081-92a3-b4c5d6e7f809
Content-Type: application/json
{
    "decision": "APPROVED",
    "justification": "All 611 GATE criteria PASS; 1 FAIL remediated under RCA-2026-08-09-A; see attachment.",
    "na_justifications": []
}

```

Responses

Status	Description
201	Sign-off recorded. The response body is the SignOff object.
400	Malformed request body.

401	Missing or invalid bearer token.
403	Token lacks the certification:signoff scope, or the account is not designated as an RCA.
404	No release with this ID exists.
409	The release already has a sign-off; revoke first to amend.
422	The decision is APPROVED but one or more GATE criteria FAIL.
429	Rate limit exceeded.

Security

Requires the ``certification:signoff`` scope (certifier only).

Example Request

```
POST /v1/certification/releases/REL-2026-08-09-001/signoff HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Idempotency-Key: 1a2b3c4d-5e6f-7081-92a3-b4c5d6e7f809
Content-Type: application/json
{  "decision":  "APPROVED",  "justification":  "...",  "na_justifications":  []  }
```

Example Response

```
HTTP/1.1 201 Created
Location: /v1/certification/releases/REL-2026-08-09-001/signoff/SO-2026-08-09-001
Content-Type: application/json
{
  "id": "SO-2026-08-09-001",
  "release_id": "REL-2026-08-09-001",
  "decision": "APPROVED",
  "signed_by": "rca@opuspublica.com",
  "signed_at": "2026-08-09T02:45:00.000Z",
  "justification": "..."
}
```

Error Cases

- **signoff:gate-blocked** — Returned as 422 when ``decision`` is ``APPROVED`` but at least one GATE criterion FAILs.
- **signoff:already-signed** — Returned as 409 when a sign-off already exists for this release.

Chapter 7 — Endpoint Reference: SLO Service

The SLO Service exposes the SLO definitions and the live SLI values that the governance dashboard renders. SLOs are defined in Volume II Chapter 32 of the RC1 Constitution; this service reads them from the MRC and joins them with the live measurements collected by Prometheus. The service is read-only.

7.1 GET /v1/slos — List SLOs

GET /v1/slos — *List all SLO definitions.*

Description

Returns a paginated list of all SLOs defined in the constitution, with their target, window, and current health. The list is the read path for the dashboard's SLO panel and for alerting on exhausted error budgets.

Parameters

Name	In	Required	Type	Description
health	query	no	string (enum)	Filter by current health (HEALTHY, AT_RISK, EXHAUSTED).
owner	query	no	string	Filter by owning team.
cursor	query	no	string	Pagination cursor.
page_size	query	no	integer	Page size; default 50, max 200.

Responses

Status	Description
200	Paginated list of SLO objects.
400	Invalid query parameter.
401	Missing or invalid bearer token.
403	Token lacks the slo:read scope.
429	Rate limit exceeded.

Security

Requires the ``slo:read`` scope.

Example Request

```
GET /v1/slos?health=EXHAUSTED HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "data": [
    {
      "id": "SLO-DOI-RESOLUTION",
      "name": "DOI resolution success rate",
      "target": 0.999,
      "window_days": 28,
      "owner": "platform-team",
      "current_sli": 0.9986,
      "health": "AT_RISK",
      "error_budget_remaining": 0.42
    }
  ],
  "next_cursor": null,
  "has_more": false
}
```

7.2 GET /v1/slos/{id}/status — Current SLI Value and Error Budget

GET /v1/slos/{id}/status — Get the current SLI value and error budget state for a single SLO.

Description

Returns the live SLI value, the error budget consumed and remaining, the current health classification (HEALTHY, AT_RISK, EXHAUSTED), and the timestamp of the last measurement. This endpoint is polled by the dashboard every 30 seconds.

Parameters

Name	In	Required	Type	Description
id	path	yes	string	The SLO ID (e.g., ``SLO-DOI-RESOLUTION``).

Responses

Status	Description
200	An SLOStatus object.
401	Missing or invalid bearer token.
403	Token lacks the slo:read scope.
404	No SLO with this ID exists.
429	Rate limit exceeded.

Security

Requires the ``slo:read`` scope.

Example Request

```
GET /v1/slos/SLO-DOI-RESOLUTION/status HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "slo_id": "SLO-DOI-RESOLUTION",
    "current_sli": 0.9986,
    "target": 0.999,
    "window_days": 28,
    "error_budget_total": 3600,
    "error_budget_consumed": 2088,
    "error_budget_remaining": 1512,
    "error_budget_remaining_ratio": 0.42,
    "health": "AT_RISK",
    "measured_at": "2026-08-09T02:30:00.000Z"
}
```

7.3 GET /v1/slos/{id}/burn-rate — Burn Rate History

GET /v1/slos/{id}/burn-rate — Get the burn-rate history for an SLO.

Description

Returns the burn-rate history for the named SLO over the requested time range. Burn rate is the rate at which the error budget is being consumed, expressed as a multiple of the sustainable rate (1.0 =

sustainable; 2.0 = consuming the budget twice as fast as sustainable). The SRE workbook's multi-window multi-burn-rate alerting thresholds (e.g., 14.4x over 1h, 6x over 6h) are computed from this data.

Parameters

Name	In	Required	Type	Description
id	path	yes	string	The SLO ID.
from	query	no	string (date-time)	Start of the range; default 24 hours ago.
to	query	no	string (date-time)	End of the range; default now.
resolution	query	no	string (enum)	Bucket size (`1m`, `5m`, `1h`, `1d`). Default `5m`.

Responses

Status	Description
200	An array of BurnRatePoint objects.
400	Invalid query parameter (e.g., `from` after `to`).
401	Missing or invalid bearer token.
403	Token lacks the slo:read scope.
404	No SLO with this ID exists.
429	Rate limit exceeded.

Security

Requires the `slo:read` scope.

Example Request

```
GET /v1/slos/SLO-DOI-RESOLUTION/burn-rate?resolution=1h HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "slo_id": "SLO-DOI-RESOLUTION",
  "resolution": "1h",
}
```

```
      "points": [
    { "timestamp": "2026-08-09T01:00:00.000Z", "burn_rate": 0.8, "window": "1h" },
    { "timestamp": "2026-08-09T02:00:00.000Z", "burn_rate": 2.3, "window": "1h" }
  ]
}
```

Error Cases

- **slo:range-too-wide** — The requested range exceeds 30 days; narrow the range.

Chapter 8 — Endpoint Reference: Self-Audit Service

The Self-Audit Service exposes the outputs of the Self-Auditing Platform (SAP): the nightly audit report, active findings, ML-detected anomalies, and remediation records. The SAP runs the NIST SP 800-137 ISCM six-phase loop (maintain, collect, analyze, report, respond, review) every night and writes its outputs to a PostgreSQL store; this service reads that store and accepts remediation records from operators.

8.1 GET /v1/audit/nightly — Latest Nightly Audit Report

GET /v1/audit/nightly — *Get the most recent nightly audit report.*

Description

Returns the latest nightly audit report. The report summarises the previous 24 hours: integrity-rule violations, drift detections, stale evidence, anomalies above threshold, and the per-pillar health roll-up. If ``date`` is supplied, returns the report for that date instead.

Parameters

Name	In	Required	Type	Description
date	query	no	string (date)	Return the report for this date (UTC). Default: the most recent report.

Responses

Status	Description
200	A NightlyReport object.
401	Missing or invalid bearer token.
403	Token lacks the audit:read scope.
404	No report exists for the requested date.
429	Rate limit exceeded.

Security

Requires the ``audit:read`` scope.

Example Request

GET	/v1/audit/nightly	HTTP/1.1
Host:		gov-api.opuspublica.com

Authorization:	Bearer	<token>
----------------	--------	---------

Example Response

```

HTTP/1.1                200                OK
Content-Type:            application/json
{
  "report_id":            "AR-NIGHTLY-2026-08-09",
  "date":                "2026-08-09",
  "started_at":          "2026-08-09T02:00:00.000Z",
  "finished_at":         "2026-08-09T02:11:42.000Z",
  "integrity":           {
    "passed":            7,
    "failed":            1
  },
  "drift":               {
    "detected":          2,
    "samples":           1384
  },
  "stale_evidence":      4,
  "anomalies_above_threshold": 1,
  "pillar_health":       {
    "MRC":               "GREEN",
    "ATG":                "AMBER",
    "CCE":               "GREEN",
    "Dashboard":          "GREEN",
    "SAP":                "GREEN"
  }
}

```

8.2 GET /v1/audit/findings — List Active Findings**GET /v1/audit/findings** — *List active audit findings, with filters.***Description**

Returns a paginated list of active findings. A finding is a confirmed defect discovered by the SAP: an integrity-rule violation, a drift detection, a stale evidence record, or an anomaly that crossed the threshold. Each finding has a severity, a type, the affected resource, and a remediation status.

Parameters

Name	In	Required	Type	Description
severity	query	no	string (enum)	Filter by severity (SEV-1, SEV-2, SEV-3).
type	query	no	string (enum)	Filter by finding type (INTEGRITY_VIOLATION, DRIFT, STALE_EVIDENCE, ANOMALY).
status	query	no	string (enum)	Filter by status (OPEN, REMEDIATING, RESOLVED). Default

				OPEN.
cursor	query	no	string	Pagination cursor.
page_size	query	no	integer	Page size; default 50, max 200.

Responses

Status	Description
200	Paginated list of Finding objects.
400	Invalid query parameter.
401	Missing or invalid bearer token.
403	Token lacks the audit:read scope.
429	Rate limit exceeded.

Security

Requires the ``audit:read`` scope.

Example Request

```
GET /v1/audit/findings?severity=SEV-1&status=OPEN HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "data": [
    {
      "id": "F-2026-08-09-001",
      "severity": "SEV-1",
      "type": "INTEGRITY_VIOLATION",
      "rule": "IR-02",
      "resource": "MOD-legacy-importer",
      "detail": "Module has no VERIFIED_BY edge.",
      "detected_at": "2026-08-09T02:05:11.000Z",
      "status": "OPEN"
    }
  ],
  "next_cursor": null,
  "has_more": false
}
```

}

8.3 POST /v1/audit/findings/{id}/remediate — Record Remediation

/v1/audit/findings/{id}/remediate — Record the remediation of a finding.

Description

Records that the named finding has been remediated. The request body contains the root cause, the remediation action, the pull request or commit that fixed it, and the re-verification result. On success, the finding's status moves from OPEN to REMEDIATING (if re-verification is pending) or RESOLVED (if re-verification PASSES).

Request Body

The request body is a JSON object describing the remediation.

```
POST /v1/audit/findings/F-2026-08-09-001/remediate HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Idempotency-Key: 4c5d6e7f-8091-92a3-b4c5-d6e7f8091a2b
Content-Type: application/json
{
  "root_cause": "Module MOD-legacy-importer was never linked to its tests in the graph.",
  "remediation": "Added VERIFIED_BY edge from MOD-legacy-importer to T-legacy-importer
in PR #4231.",
  "pr_uri": "https://git.opuspublica.com/opus/gov/pull/4231",
  "reverification": { "rule": "IR-02", "result": "PASS", "evaluated_at": "2026-08-09T03:00:00.000Z" }
}
```

Responses

Status	Description
201	Remediation recorded. The response body is the Remediation object.
400	Malformed request body.
401	Missing or invalid bearer token.
403	Token lacks the audit:write scope.
404	No finding with this ID exists.
409	The finding is already RESOLVED.
422	The re-verification result is missing or invalid.
429	Rate limit exceeded.

Security

Requires the ``audit:write`` scope (operator, certifier, admin).

Example Request

```

POST                /v1/audit/findings/F-2026-08-09-001/remediate                HTTP/1.1
Host:                                gov-api.opuspublica.com
Authorization:                Bearer                                <token>
Idempotency-Key:                4c5d6e7f-8091-92a3-b4c5-d6e7f8091a2b
Content-Type:                application/json
{  "root_cause":  "...",  "remediation":  "...",  "pr_uri":  "...",  "reverification":
{"rule":"IR-02","result":"PASS","evaluated_at":"..."}
}
```

Example Response

```

HTTP/1.1                201                Created
Location:                /v1/audit/findings/F-2026-08-09-001/remediation/RM-2026-08-09-001
Content-Type:                application/json
{
  "id":                "RM-2026-08-09-001",
  "finding_id":                "F-2026-08-09-001",
  "recorded_by":                "operator@opuspublica.com",
  "recorded_at":                "2026-08-09T03:01:14.000Z",
  "finding_status":                "RESOLVED"
}
```

Error Cases

- **remediation:already-resolved** — Returned as 409 when the finding is already RESOLVED.

8.4 GET /v1/audit/anomalies — List ML-Detected Anomalies

GET /v1/audit/anomalies — *List anomalies detected by the ML pipeline.*

Description

Returns a paginated list of anomalies detected by the SAP's anomaly-detection pipeline. Anomalies are statistical deviations from baseline (e.g., a sudden spike in evidence-upload latency, an unusual pattern of criterion FAILs). Not all anomalies become findings; an anomaly becomes a finding only when it crosses a threshold. This endpoint returns all anomalies, including sub-threshold ones, for investigation by the on-call engineer.

Parameters

Name	In	Required	Type	Description
since	query	no	string (date-time)	Return only anomalies detected at or after this instant.
model	query	no	string	Filter by the model that detected the anomaly (e.g., ``isolation-forest-v3``).
min_score	query	no	number	Filter by anomaly score (0.0 to 1.0); default 0.5.
cursor	query	no	string	Pagination cursor.
page_size	query	no	integer	Page size; default 50, max 200.

Responses

Status	Description
200	Paginated list of Anomaly objects.
400	Invalid query parameter.
401	Missing or invalid bearer token.
403	Token lacks the audit:read scope.
429	Rate limit exceeded.

Security

Requires the ``audit:read`` scope.

Example Request

```
GET /v1/audit/anomalies?since=2026-08-09T00:00:00Z&min_score=0.7 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

HTTP/1.1	200	OK
Content-Type:		application/json
{		
	"data":	[
		{
	"id":	"AN-2026-08-09-014",
	"model":	"isolation-forest-v3",
		"score": 0.83,
"signal":	"evidence_upload_latency_p95_ms",	
	"observed_value":	4120,
	"baseline_value":	850,
"detected_at":	"2026-08-09T02:14:11.000Z",	
	"promoted_to_finding":	false
		}
		],
	"next_cursor":	null,
	"has_more":	false
}		

Chapter 9 — Endpoint Reference: Release Service

The Release Service exposes release records and the Production Acceptance Gate state, and accepts the deployment-authorization write from the CSA office. A release record is created automatically when a tag is pushed to the platform Git repository; the gate state is updated as criteria are evaluated and as the RCA signs off; the deployment-authorization is the final write that permits the deployment controller to promote the release to production.

9.1 GET /v1/releases — List Releases

GET /v1/releases — *List releases, with filters.*

Description

Returns a paginated list of releases, ordered by creation timestamp descending. Each entry contains the release ID, the SemVer tag, the Git commit SHA, the creation timestamp, the gate state, and the deployment state.

Parameters

Name	In	Required	Type	Description
gate	query	no	string (enum)	Filter by gate state (PENDING, BLOCKED, PASSED).
deployment	query	no	string (enum)	Filter by deployment state (NOT_AUTHORIZED, AUTHORIZED, DEPLOYED, ROLLED_BACK).
since	query	no	string (date-time)	Return only releases created at or after this instant.
cursor	query	no	string	Pagination cursor.
page_size	query	no	integer	Page size; default 50, max 200.

Responses

Status	Description
200	Paginated list of ReleaseRecord objects.
400	Invalid query parameter.

401	Missing or invalid bearer token.
403	Token lacks the release:read scope.
429	Rate limit exceeded.

Security

Requires the ``release:read`` scope.

Example Request

```
GET /v1/releases?gate=PASSED HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "data": [
    {
      "id": "REL-2026-08-09-001",
      "version": "2.4.1",
      "git_sha": "a1b2c3d4...",
      "created_at": "2026-08-09T01:00:00.000Z",
      "gate": "PASSED",
      "deployment": "AUTHORIZED"
    }
  ],
  "next_cursor": null,
  "has_more": false
}
```

9.2 GET /v1/releases/{id} — Get a Release Record

GET /v1/releases/{id} — Retrieve a single release record.

Description

Returns the full ReleaseRecord object for the named release, including the manifest (SBOM, SLSA provenance, signature), the gate state, the deployment state, and the URIs of the certification record and the sign-off.

Parameters

Name	In	Required	Type	Description
------	----	----------	------	-------------

id	path	yes	string	The release ID (e.g., ``REL-2026-08-09-001``).

Responses

Status	Description
200	A ReleaseRecord object.
401	Missing or invalid bearer token.
403	Token lacks the release:read scope.
404	No release with this ID exists.
429	Rate limit exceeded.

Security

Requires the ``release:read`` scope.

Example Request

```
GET /v1/releases/REL-2026-08-09-001 HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": "REL-2026-08-09-001",
  "version": "2.4.1",
  "git_sha": "a1b2c3d4e5f6...",
  "created_at": "2026-08-09T01:00:00.000Z",
  "manifest": {
    "sbom_uri": "s3://opus-sbom/REL-2026-08-09-001.cdx.json",
    "slsa_provenance_uri": "s3://opus-prov/REL-2026-08-09-001.intoto.json",
    "signature_uri": "s3://opus-sig/REL-2026-08-09-001.sig"
  },
  "gate": "PASSED",
  "deployment": "AUTHORIZED",
  "certification_record_uri": "/v1/certification/releases/REL-2026-08-09-001/record",
  "signoff_uri": "/v1/certification/releases/REL-2026-08-09-001/signoff/SO-2026-08-09-001"
```

}

9.3 GET /v1/releases/{id}/gate — Get the Production Acceptance Gate Status

GET /v1/releases/{id}/gate — *Get the Production Acceptance Gate status for a release.*

Description

Returns the gate status object: the overall state (PENDING, BLOCKED, PASSED), the per-pillar roll-up, the count of PASS/FAIL/N/A criteria, and (if BLOCKED) the list of failing criteria with their FAIL details. This endpoint is the canonical read path for the gate; the CI orchestrator checks it before issuing a deploy request.

Parameters

Name	In	Required	Type	Description
id	path	yes	string	The release ID.

Responses

Status	Description
200	An AcceptanceGate object.
401	Missing or invalid bearer token.
403	Token lacks the release:read scope.
404	No release with this ID exists.
429	Rate limit exceeded.

Security

Requires the ``release:read`` scope.

Example Request

```
GET /v1/releases/REL-2026-08-09-001/gate HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
```

```

        "release_id": "REL-2026-08-09-001",
        "state": "PASSED",
        "criteria_total": 612,
        "criteria_pass": 611,
        "criteria_fail": 0,
        "criteria_na": 1,
        "pillars": {
            "constitution": "PASS",
            "traceability": "PASS",
            "certification": "PASS",
            "slo": "PASS",
            "audit": "PASS"
        },
        "failures": []
    }

```

9.4 POST /v1/releases/{id}/deploy — Authorize Deployment

/v1/releases/{id}/deploy — Authorize the deployment of a release to production (CSA only).

Description

Authorizes the deployment controller to promote the named release to production. Authorization is permitted only when the gate state is PASSED and the deployment state is NOT_AUTHORIZED. The endpoint requires the ``release:deploy`` scope, which is granted only to the ``operator`` role (typically the CSA office or a designated deploy bot). Authorization is final: once authorized, a release cannot be un-authorized; it can only be rolled back via the rollback endpoint (not specified here).

Request Body

The request body is a JSON object containing the authorization rationale and the deployment window.

```

POST /v1/releases/REL-2026-08-09-001/deploy HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Idempotency-Key: 6e7f8091-92a3-b4c5-d6e7-f8091a2b3c4d
Content-Type: application/json
{
  "rationale": "Gate PASSED; RCA sign-off recorded; deploying in the 02:00 UTC window.",
  "window": { "start": "2026-08-09T02:00:00Z", "end": "2026-08-09T02:30:00Z" }
}

```

Responses

Status	Description
--------	-------------

201	Authorization recorded. The response body is the DeploymentAuthorization object.
400	Malformed request body.
401	Missing or invalid bearer token.
403	Token lacks the release:deploy scope, or the account is not permitted to authorize production deployments.
404	No release with this ID exists.
409	The release is already AUTHORIZED, DEPLOYED, or ROLLED_BACK.
422	The gate state is not PASSED; authorization is refused.
429	Rate limit exceeded.

Security

Requires the ``release:deploy`` scope (operator only).

Example Request

```
POST /v1/releases/REL-2026-08-09-001/deploy HTTP/1.1
Host: gov-api.opuspublica.com
Authorization: Bearer <token>
Idempotency-Key: 6e7f8091-92a3-b4c5-d6e7-f8091a2b3c4d
Content-Type: application/json
{
  "rationale": "...",
  "window": { "start": "...", "end": "..." }
}
```

Example Response

```
HTTP/1.1 201 Created
Location: /v1/releases/REL-2026-08-09-001/deploy/DA-2026-08-09-001
Content-Type: application/json
{
  "id": "DA-2026-08-09-001",
  "release_id": "REL-2026-08-09-001",
  "authorized_by": "csa@opuspublica.com",
  "authorized_at": "2026-08-09T01:55:00.000Z",
  "window": { "start": "2026-08-09T02:00:00Z", "end": "2026-08-09T02:30:00Z" }
}
```

Error Cases

- **deploy:gate-not-passed** — Returned as 422 when the gate state is not PASSED.

- **deploy:already-authorized** — Returned as 409 when the deployment state is already AUTHORIZED or later.

Chapter 10 — Events and Webhooks

The Governance API emits events whenever the governance state changes: a criterion passes or fails, an integrity rule is violated, a traceability gap is detected, evidence becomes stale, a release is blocked or authorized. Events are delivered to subscribers via webhooks. Every event is a CloudEvents 1.0 envelope; every delivery is HMAC-signed; every delivery is at-least-once with retry and dead-letter handling. This chapter specifies the event catalogue, the payload schema, the subscription API, the delivery contract, and a worked example of a CI consumer.

10.1 Event Types Emitted

The Governance API emits the following event types. Each ``type`` is a reverse-DNS identifier following the CloudEvents naming guidance. The ``source`` is always ``/opuspublica/governance``.

Type	Emitted When	Severity
opuspublica.governance.criterion.passed	A criterion evaluated to PASS.	INFO
opuspublica.governance.criterion.failed	A criterion evaluated to FAIL.	SEV-1 if GATE, SEV-3 if INFO
opuspublica.governance.invariant.violated	An invariant monitor detected a violation.	SEV-1
opuspublica.governance.traceability.gap-detected	An integrity rule detected a missing edge.	SEV-2
opuspublica.governance.evidence.stale	An evidence artifact crossed its ``expires_at``.	SEV-2
opuspublica.governance.release.blocked	A release's gate moved to BLOCKED.	SEV-1
opuspublica.governance.release.authorized	A release was authorized for deployment.	INFO
opuspublica.governance.signoff.recorded	An RCA sign-off was recorded.	INFO
opuspublica.governance.anomaly.promoted	An anomaly was promoted to a finding.	SEV-2
opuspublica.governance.constitution.amended	The constitution was amended (new version published).	INFO
opuspublica.governance.slo.exhausted	An SLO's error budget was exhausted.	SEV-1
opuspublica.governance.slo.at-risk	An SLO crossed the AT_RISK threshold.	SEV-2

10.2 Event Payload Schema (CloudEvents 1.0)

Every event is a CloudEvents 1.0 envelope in JSON encoding. The envelope carries the CloudEvents mandatory attributes (`id`, `source`, `specversion`, `type`, `time`) plus the Opus Publica extension attributes `trace_id` and `traceparent` for distributed tracing. The event `data` is the domain-specific payload (a Criterion, an Invariant, an EvidenceArtifact, etc.).

```
Content-Type: application/cloudevents+json
{
  "specversion": "1.0",
  "id": "9d25f46b-dafb-5789-b941-b7177a87c95a",
  "source": "/opuspublica/governance",
  "type": "opuspublica.governance.criterion.failed",
  "subject": "CRT-INV-I-01-a",
  "time": "2026-08-09T02:15:33.001Z",
  "datacontenttype": "application/json",
  "trace_id": "4f5e6d7c8b9a0f1e2d3c4b5a6c7d8e9f",
  "traceparent": "00-4f5e6d7c8b9a0f1e2d3c4b5a6c7d8e9f-1234567890abcdef-01",
  "data": {
    "criterion_id": "CRT-INV-I-01-a",
    "provision_id": "INV-I-01",
    "release_id": "REL-2026-08-09-001",
    "result": "FAIL",
    "gating": "GATE",
    "evidence": ["EV-2026-08-09-002"],
    "evaluated_at": "2026-08-09T02:15:33.001Z",
    "rego_decision_id": "4f5e6d7c-8b9a-0f1e-2d3c-4b5a-6c7d-8e9f"
  }
}
```

10.3 Webhook Subscription

Webhook subscriptions are managed through `POST /v1/webhooks` (create), `GET /v1/webhooks` (list), and `DELETE /v1/webhooks/{id}` (revoke). Creating a subscription requires the `webhook:admin` scope (admin only). The request body specifies the callback URL, the event types to subscribe to, the HMAC secret used to sign deliveries, and an optional filter expression. The HMAC secret is generated by the server and returned once in the create response; the client **MUST** store it securely (it is not retrievable thereafter).

POST	/v1/webhooks	HTTP/1.1
Host:		gov-api.opuspublica.com
Authorization:	Bearer	<token>
Idempotency-Key:		7f8091a2-b3c4-d5e6-f708-192a3b4c5d6e

```

Content-Type: application/json
{
  "callback_url": "https://ci.opuspublica.com/gov-webhook",
  "event_types": [
    "opuspublica.governance.criterion.failed",
    "opuspublica.governance.release.blocked"
  ],
  "filter": {
    "release_id": "REL-2026-08-09-*"
  }
}

HTTP/1.1 201 Created
Content-Type: application/json
{
  "id": "WH-2026-08-09-001",
  "callback_url": "https://ci.opuspublica.com/gov-webhook",
  "event_types": [
    "opuspublica.governance.criterion.failed",
    "opuspublica.governance.release.blocked"
  ],
  "hmac_secret": "c8f5e1d2a9b7c4e6f3a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3",
  "created_at": "2026-08-09T01:00:00.000Z",
  "active": true
}

```

HMAC verification. Every delivery is signed with HMAC-SHA256 using the subscription's ``hmac\_secret``. The signature is sent in the ``X-Opus-Signature`` header as ``sha256=<hex>`` over the raw request body. The receiver SHALL recompute the HMAC over the body and SHALL reject any delivery whose signature does not match. The receiver SHALL also verify the ``X-Opus-Webhook-Id`` header matches a known subscription, to prevent spoofed deliveries.

10.4 Webhook Delivery

Delivery is at-least-once. The Governance API retries failed deliveries using exponential backoff: 1 minute, 5 minutes, 30 minutes, 2 hours, 6 hours, 24 hours. A delivery is considered successful when the receiver responds with any 2xx status within 30 seconds. A delivery is considered failed on 4xx (after which the subscription is disabled) or on the 6th consecutive 5xx (after which the delivery is moved to the dead-letter queue).

Attempt	Backoff	Action on Failure
1	immediate	Retry in 1 minute.
2	1 minute	Retry in 5 minutes.
3	5 minutes	Retry in 30 minutes.
4	30 minutes	Retry in 2 hours.

5	2 hours	Retry in 6 hours.
6	6 hours	Retry in 24 hours.
7	24 hours	Move to dead-letter; disable subscription.

Dead-lettered deliveries are retained for 30 days and can be retrieved (and replayed) via ``GET /v1/webhooks/{id}/dead-letter`` and ``POST /v1/webhooks/{id}/dead-letter/{did}/replay``. Replay reissues the delivery to the original callback URL.

10.5 Worked Example: A CI Webhook Consumer that Blocks a Deploy

Consider a CI orchestrator that, when a release's gate moves to BLOCKED, must immediately halt any in-progress deploy of that release. The orchestrator subscribes to ``opuspublica.governance.release.blocked`` events. The following Python handler (using the Flask framework) verifies the HMAC signature, parses the CloudEvent, and cancels the in-progress deploy.

```
# deploy_blocker.py — CI webhook consumer.
import hmac, hashlib, json
from flask import Flask, request, abort

app = Flask(__name__)
WH_HMAC_SECRET = open("/etc/secrets/wh_hmac.key", "rb").read()

@app.route("/gov-webhook", methods=["POST"])
def gov_webhook():
    # 1. Verify HMAC signature.
    sig = request.headers.get("X-Opus-Signature", "")
    if not sig.startswith("sha256="):
        abort(401)

    expected = hmac.new(WH_HMAC_SECRET, request.get_data(),
                        hashlib.sha256).hexdigest()
    if not hmac.compare_digest(sig[len("sha256="):], expected):
        abort(401)

    # 2. Parse the CloudEvent.
    event = request.get_json()
    if event["type"] != "opuspublica.governance.release.blocked":
        return "", 200 # not the event we subscribed to (filter mismatch)

    # 3. Cancel any in-progress deploy of the named release.
    release_id = event["data"]["release_id"]
    cancel_deploy(release_id)
    return "", 200

def cancel_deploy(release_id):
```

```
# Implementation: cancel the in-progress deploy via the deploy controller.
...
if __name__ == "__main__":
    app.run(port=8080)
```

The handler returns ``200`` even for events it does not act on, to avoid being disabled for 4xx responses. It performs the HMAC verification before any other work and uses ``hmac.compare\_digest`` to prevent timing attacks. The release-blocking behaviour depends only on the event type and the ``release\_id`` in the ``data`` field; the handler does not query the API for additional state, which keeps the blocking decision fast and local.

Chapter 11 — SDKs and Client Libraries

The Governance API is shipped with three officially supported SDKs (Python, TypeScript, Go) and a CLI. The SDKs are generated from the OpenAPI 3.1 document and add a thin hand-written layer for authentication, retry, pagination, and idempotency. The CLI is built on top of the Python SDK. All four artifacts are versioned in lockstep with the API major version: a Python SDK version 1.x targets API v1; a Python SDK version 2.x targets API v2.

11.1 The Python SDK

The Python SDK is published to PyPI as `opus-governance`. It targets Python 3.10+ and has no required runtime dependencies beyond `httpx` and `pydantic`. The SDK exposes one client class per service (`ConstitutionClient`, `TraceabilityClient`, `CertificationClient`, `SLOClient`, `AuditClient`, `ReleaseClient`, `WebhookClient`) plus a top-level `GovernanceClient` that aggregates them. Authentication is handled by an `OAuth2TokenProvider` that caches tokens and refreshes them proactively before expiry.

Installation

```
$ pip install opus-governance
```

Authentication

```
from opus_governance import OAuth2TokenProvider, GovernanceClient

token_provider = OAuth2TokenProvider(
    token_url="https://auth.opuspublica.com/oauth/token",
    client_id="ci-orchestrator@opuspublica",
    client_secret=open("/etc/secrets/client_secret").read().strip(),
    scopes=["constitution:read", "traceability:read", "traceability:write",
           "certification:evaluate", "release:read"],
)

client = GovernanceClient(
    environment="production",
    token_provider=token_provider,
)
```

Basic Usage

```
# List all SEV-1 invariants.
provisions = client.constitution.list_provisions(
    class_="INVARIANT", severity="SEV-1"
)

for p in provisions.data:
```



```

                                print(p.id,                p.title)

#    Get    the    13-node    traceability    chain    for    a    provision.
chain    =    client.traceability.get_chain("INV-I-01",    include_evidence=True)
print(f" {len(chain.nodes)}    nodes;    integrity    passed={chain.integrity.passed}")

#            Evaluate            a            criterion            synchronously.
result            =            client.certification.evaluate(
                                criterion_id="CRT-INV-I-01-a",
                                input_={"work_id":            "W-12345"},
)
print(f" {result.criterion_id}:                                {result.result}")

```

Idempotent Writes

```

import                                                                uuid

#    Upload    evidence    with    an    idempotency    key    (auto-generated    by    default).
artifact            =            client.traceability.upload_evidence(
                                evidence_type="scan",
                                provision_id="INV-I-01",
                                edge_id="MOD-canonical->T-canonical-01",
                                expires_at="2026-08-09T03:00:00Z",
                                file_bytes=open("scan.json",            "rb").read(),
                                idempotency_key=str(uuid.uuid4()),
)
print(artifact.id,                                                                artifact.sha256)

```

11.2 The TypeScript SDK

The TypeScript SDK is published to npm as ``@opuspublica/governance``. It targets Node.js 18+ and modern browsers (it uses ``fetch``). The SDK is generated with ``openapi-typescript`` for types and ``openapi-fetch`` for the HTTP layer. It exposes one client per service like the Python SDK, and handles authentication, retry, pagination, and idempotency identically.

Installation

```
$ npm install @opuspublica/governance
```

Authentication

```

import { OAuth2TokenProvider, GovernanceClient } from "@opuspublica/governance";

const tokenProvider = new OAuth2TokenProvider({

```

```

        tokenUrl: "https://auth.opuspublica.com/oauth/token",
        clientId: "ci-orchestrator@opuspublica",
        clientSecret: process.env.OP_GOV_CLIENT_SECRET!,
        scopes: ["constitution:read", "traceability:read", "release:read"],
    });

    const client = new GovernanceClient({
        environment: "production",
        tokenProvider,
    });

```

Basic Usage

```

// List all SEV-1 invariants.
const provisions = await client.constitution.listProvisions({
    class: "INVARIANT",
    severity: "SEV-1",
});

for (const p of provisions.data) {
    console.log(p.id, p.title);
}

// Get the traceability chain for a provision.
const chain = await client.traceability.getChain("INV-I-01", { includeEvidence: true });
console.log(`${chain.nodes.length} nodes; integrity passed=${chain.integrity.passed}`);

// Evaluate a criterion synchronously.
const result = await client.certification.evaluate({
    criterionId: "CRT-INV-I-01-a",
    input: { work_id: "W-12345" },
});
console.log(`${result.criterion_id}: ${result.result}`);

```

11.3 The Go SDK

The Go SDK is published as the module `github.com/opuspublica/governance-go`. It targets Go 1.22+ and is generated with `oapi-codegen` from the OpenAPI document. It exposes the same per-service client surface as the Python and TypeScript SDKs. The Go SDK takes a `context.Context` on every method to honor cancellation and timeouts, and it returns typed errors that wrap the underlying `Problem`.

Installation

```
$ go get github.com/opuspublica/governance-go@latest
```

Authentication and Basic Usage

```

package main

import (
    "context"
    "fmt"
    "os"

    "github.com/opuspublica/governance-go"
)

func main() {
    tokenProvider := governance.NewOAuth2TokenProvider(
        "https://auth.opuspublica.com/oauth/token",
        "ci-orchestrator@opuspublica",
        os.Getenv("OP_GOV_CLIENT_SECRET"),
        []string{"constitution:read", "traceability:read", "release:read"},
    )
    client, err := governance.NewClient(governance.EnvironmentProduction, tokenProvider)
    if err != nil {
        panic(err)
    }

    ctx := context.Background()
    provisions, err := client.Constitution.ListProvisions(ctx,
        &governance.ListProvisionsParams{Class: governance.Ptr("INVARIANT"), Severity:
governance.Ptr("SEV-1")})
    if err != nil {
        panic(err)
    }
    for _, p := range provisions.Data {
        fmt.Println(p.Id, p.Title)
    }
}

```

11.4 The CLI

The CLI is published as ``opus-gov`` (PyPI) and as a single static binary (``opus-gov`` for Linux/macOS, ``opus-gov.exe`` for Windows) built with PyInstaller. The CLI is a thin wrapper over the Python SDK; it accepts credentials via environment variables (``OP\_GOV\_CLIENT\_ID``, ``OP\_GOV\_CLIENT\_SECRET``, ``OP\_GOV\_ENV``) or via the ``--client-id`` / ``--client-secret`` / ``--env`` flags. Every command corresponds to a single API call; the output is JSON by default and can be filtered with ``--jq`` for ad-hoc scripting.

Installation

```

$ pip install opus-gov
# or, for the static binary:
$ curl -sSL https://opuspublica.com/cli/install | sh

```

Common Commands

```

# List all SEV-1 invariants.
$ opus-gov constitution list --class INVARIANT --severity SEV-1

# Get the traceability chain for a provision.
$ opus-gov traceability chain INV-I-01 --include-evidence

# Evaluate a criterion synchronously.
$ opus-gov certification evaluate CRT-INV-I-01-a --input '{"work_id":"W-12345"}'

# Get the production acceptance gate for a release.
$ opus-gov release gate REL-2026-08-09-001

# Subscribe a webhook.
$ opus-gov webhook create \
  --callback-url https://ci.opuspublica.com/gov-webhook \
  --event-type opuspublica.governance.criterion.failed \
  --event-type opuspublica.governance.release.blocked

# Pipe a list of failing criteria through jq.
$ opus-gov certification results --release-id REL-2026-08-09-001 --result FAIL \
  | jq '.data[].criterion_id'

```

Chapter 12 — Counterarguments and Limitations

This chapter addresses the principal limitations of the Governance API as specified and the principal counterarguments raised against the design. The treatment is deliberately candid: a specification that does not address its own weaknesses is a specification that cannot be trusted to govern itself.

12.1 Limitation: The API Adds a Dependency

Every tool that consumes the Governance API — CI, dashboards, auditors, IDE plugins — depends on the API's availability. An API outage becomes a governance outage: a CI pipeline that cannot evaluate criteria cannot decide whether to deploy; a dashboard that cannot read SLOs cannot display health; an auditor that cannot read findings cannot certify. This is the converse of the API's strength: by centralising governance, the API also centralises its failure mode.

Mitigation. The API is deployed with a 99.9% monthly uptime SLA in production and is engineered for graceful degradation. Reads against the Constitution and Traceability services are served from a read replica with a 30-second freshness guarantee, so an outage of the primary does not block reads. The CI orchestrator SHALL implement a fail-open policy for read-only checks: if the API is unavailable for more than 60 seconds, the orchestrator proceeds with a previously-cached certification decision (stamped with the cache timestamp) and emits a ``GovernanceAPIUnavailable`` event that the SAP flags. Write operations (evidence upload, sign-off, deployment authorization) are not fail-open; they queue locally and retry. A long API outage (more than 4 hours) is a SEV-1 incident with on-call escalation to the CSA.

12.2 Limitation: The API Surface Is Large

This specification defines 25+ endpoints across seven services; the full surface with sub-resources exceeds 50 endpoints. A large surface is a maintenance burden: the SDKs must keep up; the documentation must stay accurate; the deprecation policy must be enforced; the test matrix grows combinatorially. Each new endpoint is also a new attack surface, a new place for an authorization bug, and a new place where backward compatibility can be broken inadvertently.

Mitigation. The surface is bounded by the seven-service taxonomy; no new service SHALL be added without CSA approval and a written justification. Within each service, new endpoints SHALL be additive only (no new verb on an existing path without a major version bump). The SDKs are auto-generated from the OpenAPI document, which keeps the maintenance burden proportional to the surface (a new endpoint is one entry in the OpenAPI document, not a manual change in three SDKs). A nightly contract test verifies that the live API matches the OpenAPI document; any drift is a SEV-2 finding.

12.3 Limitation: Synchronous Evaluation May Be Too Slow

The ``POST /v1/certification/evaluate`` endpoint compiles and runs a Rego query synchronously, with a 5-second timeout. For complex criteria over large inputs, this is too slow: a criterion that joins the full

traceability graph may take 30 seconds or more. CI pipelines that need to evaluate many criteria per build can stack up to minutes of latency, which is unacceptable for fast-feedback workflows.

Mitigation. The endpoint is intended for testing and for criteria whose Rego rules are $O(n)$ or better; criteria with worse complexity SHALL be evaluated asynchronously via the certification run workflow (not specified here, but see the RC2 Roadmap Chapter 4). The Rego rule cache is warmed on bundle load; typical criteria return in under 100 milliseconds. CI pipelines SHALL batch evaluations in parallel (the rate limit of 30 per minute is per service account, not per process), and SHALL fall back to the asynchronous workflow if a criterion times out twice in a row.

12.4 Counterargument: “Why Not Just Query Neo4j Directly?”

The traceability data lives in Neo4j. Why not give integrations a Neo4j Bolt connection and let them query it directly? The answer has four parts. First, direct database access breaks encapsulation: integrations become coupled to the internal schema, and any schema migration becomes a breaking change. Second, direct access bypasses authorization: Neo4j's role-based access control does not map cleanly to the governance scopes (`traceability:read` vs. `traceability:write`). Third, direct access bypasses audit: the API records every read in the audit log, but a raw Cypher query does not. Fourth, direct access breaks the technology-neutrality commitment of the Constitution: a future migration from Neo4j to a different graph store (or to a SQL-backed graph) would break every integration. The API is the abstraction that lets the underlying store evolve without breaking consumers.

12.5 Counterargument: “Why Not GraphQL?”

GraphQL would let integrations request exactly the fields they need, eliminating over-fetching and under-fetching. Why not adopt GraphQL instead of REST? The answer has three parts. First, GraphQL's flexibility is its weakness for governance: a GraphQL query that joins every provision to every evidence is trivially expressible, and there is no clean way to rate-limit per resource. The REST surface, by contrast, exposes a bounded set of operations whose cost is predictable. Second, GraphQL's introspection makes the schema a discovery surface for attackers, requiring additional care to suppress sensitive fields; the REST surface is opaque by default. Third, the OpenAPI 3.1 ecosystem (code generators, linters, contract testers, mock servers) is more mature than the GraphQL ecosystem for the kind of fine-grained schema evolution governance requires. The cost of GraphQL's flexibility exceeds its benefit for this API. (A GraphQL facade MAY be explored as a future evolution; see Chapter 13.)

Chapter 13 — Conclusion and Implications

13.1 Summary

This specification has defined the Governance API: the single read-write surface of the RC2 governance system, exposing the five pillars (MRC, ATG, CCE, dashboard, SAP) as a versioned, OAuth-2.0-authenticated, RBAC-authorized, RFC 7807-error-formatted, cursor-paginated, idempotent, OpenAPI 3.1-described HTTPS API. The specification has covered: the architecture and conventions (Chapters 1 through 3); the 25+ endpoints across seven services (Chapters 4 through 9); the CloudEvents 1.0 event catalogue and webhook subscription and delivery contract (Chapter 10); the Python, TypeScript, and Go SDKs and the CLI (Chapter 11); and the candid limitations and counterarguments (Chapter 12). The full OpenAPI 3.1 document, error catalog, scope-permission matrix, and references are in the appendices.

13.2 Implications for Integration

The implications of this specification for integration authors are direct. First, every governance integration — CI pipeline, dashboard, auditor script, IDE plugin — SHALL consume the API through one of the official SDKs or the CLI. Direct database access, file-system reads of the MRC, and ad-hoc HTTP against unspecified endpoints are prohibited. Second, every integration SHALL handle the full error catalogue (Appendix B) and the rate-limit contract; the SDKs do this by default, but raw HTTP clients must implement it explicitly. Third, every integration that performs writes SHALL use the ``Idempotency-Key`` header on every request; the SDKs generate one by default. Fourth, every integration that subscribes to events SHALL verify the HMAC signature and SHALL treat the absence of a valid signature as a spoofed delivery. Fifth, every integration SHALL be parameterised by environment; hard-coding the production base URL is a defect.

The implication for the governance system implementers is equally direct. The OpenAPI document in Appendix A SHALL be the canonical contract; the live API SHALL match it exactly, verified by the nightly contract test. Any drift is a SEV-2 finding. Backward-incompatible changes SHALL be gated behind a major version increment and SHALL be released alongside the previous version for the deprecation window.

13.3 Future Evolution

Three evolutionary directions are anticipated. First, a GraphQL facade (Chapter 12 §12.5) MAY be added in a future minor version, exposed at ``/v1/graphql`` and backed by the same data and the same authorization model. The facade would not replace the REST surface; it would supplement it for integrations that benefit from field-level selection. Second, a streaming interface (Server-Sent Events or gRPC streaming) MAY be added for the SLO burn-rate and audit anomaly endpoints, replacing the current poll-based pattern. Third, federation with peer governance systems (other Opus Publica instances, scholarly-publishing consortia) MAY be introduced through a peer-to-peer event relay,

allowing one instance's release events to gate another instance's deployments. These directions are speculative; they are documented here so that future readers know they were considered and so that the current API is designed to accommodate them without breaking changes.

Appendix A — Full OpenAPI 3.1 Specification (Consolidated)

This appendix reproduces the consolidated OpenAPI 3.1 document for the Governance API. The document is also published as ``governance-api.yaml`` in the governance Git repository and is the canonical artifact used by the SDK generators, the contract tester, and the API gateway. The reproduction here is necessarily truncated for length; the full document exceeds 2,000 lines. The excerpt below covers the top-level structure, the security scheme, the reusable responses, the central schemas, and one representative endpoint per service.

```

openapi: 3.1.0
jsonSchemaDialect: https://json-schema.org/draft/2020-12/schema
info:
  title: Opus Publica Governance API
  version: 1.0.0
  description: |
    The read-write surface of the RC2 governance system. Every governance primitive
    is addressable by a stable URI; every mutation is authenticated, authorized,
    idempotent, and immutable. See the Governance API Specification (OP-RC2-API-001)
    for the normative contract; this document is the machine-readable form.
  contact:
    name: Office of the Chief Systems Architect
    email: gov-api@opuspublica.com
  license:
    name: Apache 2.0
    url: https://www.apache.org/licenses/LICENSE-2.0.html
servers:
  - url: https://gov-api.dev.opuspublica.com/v1
    description: Development
  - url: https://gov-api.staging.opuspublica.com/v1
    description: Staging
  - url: https://gov-api.opuspublica.com/v1
    description: Production
tags:
  - name: Constitution
  - name: Traceability
  - name: Certification
  - name: SLO
  - name: Audit
  - name: Release
  - name: Webhooks
security:
  - OAuth2: []
paths:
  /constitution:

```

```

get:
  tags: [Constitution]
  operationId: listProvisions
summary: List constitutional provisions.
parameters:
  - $ref: '#/components/parameters/PageSize'
  - $ref: '#/components/parameters/Cursor'
  - name: class
    in: query
schema: { $ref: '#/components/schemas/ProvisionClass' }
  - name: severity
    in: query
schema: { $ref: '#/components/schemas/Severity' }
  - name: owner
    in: query
  schema: { type: string }
  - name: status
    in: query
schema: { $ref: '#/components/schemas/ProvisionStatus' }
responses:
  '200':
description: Paginated list of provisions.
content:
  application/json:
    schema:
      allOf:
        - $ref: '#/components/schemas/Page'
        - type: object
          properties:
            data:
              type: array
            items: { $ref: '#/components/schemas/Provision' }
  '400': { $ref: '#/components/responses/Validation' }
  '401': { $ref: '#/components/responses/Unauthorized' }
  '403': { $ref: '#/components/responses/Forbidden' }
  '429': { $ref: '#/components/responses/RateLimited' }
security:
  - OAuth2: [constitution:read]
    /constitution/{provision_id}:
get:
  tags: [Constitution]
  operationId: getProvision
parameters:
  - name: provision_id
    in: path
    required: true
  schema: { type: string }
responses:

```

```

'200':
  description: The provision.
  content:
    application/json:
      schema: { $ref: '#/components/schemas/Provision' }
'404': { $ref: '#/components/responses/NotFound' }
security:
  - OAuth2: [constitution:read]
    /traceability/requirements/{id}/chain:
      get:
        tags: [Traceability]
        operationId: getTraceabilityChain
        parameters:
          - name: id
            in: path
            required: true
            schema: { type: string }
          - name: include_evidence
            in: query
            schema: { type: boolean, default: false }
        responses:
          '200':
            description: The 13-node traceability chain.
            content:
              application/json:
                schema: { $ref: '#/components/schemas/TraceabilityGraph' }
            security:
              - OAuth2: [traceability:read]
                /certification/evaluate:
                  post:
                    tags: [Certification]
                    operationId: evaluateCriterion
                    parameters:
                      - $ref: '#/components/parameters/IdempotencyKey'
                    requestBody:
                      required: true
                      content:
                        application/json:
                          schema:
                            type: object
                            required: [criterion_id]
                            properties:
                              criterion_id: { type: string }
                    input: { type: object, additionalProperties: true }
                    responses:
                      '200':
                        description: Evaluation result.
                        content:

```

```

                                application/json:
schema: { $ref: '#/components/schemas/AssessmentResult' }
'429': { $ref: '#/components/responses/RateLimited' }
                                security:
- OAuth2: [certification:evaluate]
                                /slos:
                                get:
                                tags: [SLO]
                                operationId: listSLOs
                                responses:
                                '200':
description: Paginated list of SLOs.
                                content:
                                application/json:
                                schema:
                                allOf:
- $ref: '#/components/schemas/Page'
- type: object
                                properties:
                                data:
                                type: array
items: { $ref: '#/components/schemas/SLO' }
                                security:
- OAuth2: [slo:read]
                                /audit/nightly:
                                get:
                                tags: [Audit]
                                operationId: getNightlyReport
                                responses:
                                '200':
description: The nightly audit report.
                                content:
                                application/json:
schema: { $ref: '#/components/schemas/NightlyReport' }
                                security:
- OAuth2: [audit:read]
                                /releases/{id}/deploy:
                                post:
                                tags: [Release]
                                operationId: authorizeDeployment
                                parameters:
- name: id
  in: path
  required: true
  schema: { type: string }
- $ref: '#/components/parameters/IdempotencyKey'
                                requestBody:
                                required: true

```

```

                                content:
                                application/json:
schema: { $ref: '#/components/schemas/DeployAuthorizationRequest' }
                                responses:
                                '201':
                                description: Authorization recorded.
                                content:
                                application/json:
schema: { $ref: '#/components/schemas/DeploymentAuthorization' }
    '422': { $ref: '#/components/responses/Validation' }
                                security:
                                - OAuth2: [release:deploy]
                                /webhooks:
                                post:
                                tags: [Webhooks]
                                operationId: createWebhook
                                parameters:
                                - $ref: '#/components/parameters/IdempotencyKey'
                                requestBody:
                                required: true
                                content:
                                application/json:
schema: { $ref: '#/components/schemas/WebhookSubscription' }
                                responses:
                                '201':
                                description: Subscription created.
                                content:
                                application/json:
schema: { $ref: '#/components/schemas/WebhookSubscription' }
                                security:
                                - OAuth2: [webhook:admin]
components:
                                securitySchemes:
                                OAuth2:
                                type: oauth2
                                flows:
                                clientCredentials:
                                tokenUrl: https://auth.opuspublica.com/oauth/token
                                scopes:
                                constitution:read: Read provisions.
                                constitution:write: Amend the constitution (admin only).
                                traceability:read: Query the graph and read evidence.
                                traceability:write: Upload evidence and create nodes.
                                certification:read: Read criteria and assessment results.
                                certification:evaluate: Evaluate a criterion.
                                certification:signoff: Record RCA sign-off.
                                slo:read: Read SLOs and SLI values.
                                audit:read: Read findings and anomalies.

```

```

        audit:write:  Record  remediations.
        release:read: Read  release  records.
        release:deploy:  Authorize  a  deployment.
        webhook:admin:  Manage  webhooks.
        account:admin:  Manage  service  accounts.
        parameters:
            PageSize:
                name:  page_size
                in:  query
schema: { type: integer, minimum: 1, maximum: 200, default: 50 }
            Cursor:
                name:  cursor
                in:  query
            schema: { type: string }
            IdempotencyKey:
                name:  Idempotency-Key
                in:  header
                required: false
schema: { type: string, format: uuid }
        responses:
            Unauthorized:
                description: Missing or invalid credentials.
                content:
                    application/problem+json:
schema: { $ref: '#/components/schemas/Problem' }
            Forbidden:
                description: Insufficient scope.
                content:
                    application/problem+json:
schema: { $ref: '#/components/schemas/Problem' }
            NotFound:
                description: Resource not found.
                content:
                    application/problem+json:
schema: { $ref: '#/components/schemas/Problem' }
            Validation:
                description: Schema validation failed.
                content:
                    application/problem+json:
schema: { $ref: '#/components/schemas/Problem' }
            RateLimited:
                description: Rate limit exceeded.
                headers:
Retry-After: { schema: { type: integer, minimum: 1 } }
                content:
                    application/problem+json:
schema: { $ref: '#/components/schemas/Problem' }
        schemas:

```

```

    Problem:
      type: object
      required: [type, title, status]
      properties:
        type: { type: string, format: uri }
        title: { type: string }
        status: { type: integer }
        detail: { type: string }
        instance: { type: string, format: uri }
        trace_id: { type: string }
      errors:
        type: array
        items:
          type: object
          properties:
            field: { type: string }
            message: { type: string }
      Page:
        type: object
        properties:
          data: { type: array, items: {} }
          next_cursor: { type: string, nullable: true }
          has_more: { type: boolean }
      ProvisionClass:
        type: string
      enum: [INVARIANT, RULE, CONTRACT, FIELD, EVENT, SLO, THRESHOLD,
        STATE_MACHINE, SECURITY, CRITERION, GATE]
      Severity:
        type: string
        enum: [SEV-1, SEV-2, SEV-3]
      ProvisionStatus:
        type: string
        enum: [ACTIVE, DEPRECATED, SUPERSEDED, DRAFT]
      Provision:
        type: object
        required: [id, class, severity, status, title, body, owner, version]
        properties:
          id: { type: string }
          class: { $ref: '#/components/schemas/ProvisionClass' }
          severity: { $ref: '#/components/schemas/Severity' }
          status: { $ref: '#/components/schemas/ProvisionStatus' }
          title: { type: string }
          body: { type: string }
          owner: { type: string }
          version: { type: string }
          source: { type: object }
          traceability_root: { type: string }
      Criterion:

```

```

      type: object
    required: [id, provision_id, gating, pass_condition, rego_rule_uri]
    properties:
      id: { type: string }
      provision_id: { type: string }
      gating: { type: string, enum: [GATE, INFO] }
      pass_condition: { type: string }
      fail_condition: { type: string }
      verification_method: { type: string }
      max_evidence_age_seconds: { type: integer }
      rego_rule_uri: { type: string, format: uri }
      verifier_role: { type: string }
  AssessmentResult:
    type: object
    required: [criterion_id, result, evaluated_at]
    properties:
      criterion_id: { type: string }
      release_id: { type: string }
      result: { type: string, enum: [PASS, FAIL, NOT_APPLICABLE] }
      evaluated_at: { type: string, format: date-time }
      evaluated_by: { type: string }
      evidence: { type: array, items: { type: string } }
      rego_decision_id: { type: string }
      duration_ms: { type: integer }
  EvidenceArtifact:
    type: object
    required: [id, evidence_type, artifact_uri, sha256, collected_at, collected_by]
    properties:
      id: { type: string }
      evidence_type: { type: string,
        enum: [scan, log, screenshot, query, reconciliation, attestation] }
      artifact_uri: { type: string, format: uri }
      sha256: { type: string, pattern: '^[a-f0-9]{64}$' }
      collected_at: { type: string, format: date-time }
      collected_by: { type: string }
      expires_at: { type: string, format: date-time }
  TraceabilityNode:
    type: object
    required: [id, type, label]
    properties:
      id: { type: string }
      type: { type: string }
      label: { type: string }
  TraceabilityEdge:
    type: object
    required: [from, to, kind]
    properties:
      from: { type: string }

```



```

        to: { type: string }
        kind: { type: string,
enum: [REALIZED_BY, IMPLEMENTED_IN, VERIFIED_BY, EVIDENCED_BY,
        CERTIFIED_BY, AUDITED_BY] }
        TraceabilityGraph:
            type: object
            properties:
                root: { type: string }
        nodes: { type: array, items: { $ref: '#/components/schemas/TraceabilityNode' } }
        edges: { type: array, items: { $ref: '#/components/schemas/TraceabilityEdge' } }
        evidence: { type: array, items: { $ref: '#/components/schemas/EvidenceArtifact' } }
        integrity: { type: object }
        SLO:
            type: object
            required: [id, name, target, window_days, owner]
            properties:
                id: { type: string }
                name: { type: string }
                target: { type: number, minimum: 0, maximum: 1 }
                window_days: { type: integer }
                owner: { type: string }
                current_sli: { type: number }
        health: { type: string, enum: [HEALTHY, AT_RISK, EXHAUSTED] }
        error_budget_remaining: { type: number }
        NightlyReport:
            type: object
            required: [report_id, date, started_at, finished_at]
            properties:
                report_id: { type: string }
                date: { type: string, format: date }
                started_at: { type: string, format: date-time }
                finished_at: { type: string, format: date-time }
                integrity: { type: object }
                drift: { type: object }
                stale_evidence: { type: integer }
                anomalies_above_threshold: { type: integer }
                pillar_health: { type: object }
        ReleaseRecord:
            type: object
            required: [id, version, git_sha, created_at, gate, deployment]
            properties:
                id: { type: string }
                version: { type: string }
                git_sha: { type: string }
                created_at: { type: string, format: date-time }
                gate: { type: string, enum: [PENDING, BLOCKED, PASSED] }
                deployment: { type: string, enum: [NOT_AUTHORIZED, AUTHORIZED, DEPLOYED,
ROLLED_BACK] }

```

```

manifest: { type: object }
DeployAuthorizationRequest:
  type: object
  required: [rationale, window]
  properties:
    rationale: { type: string }
    window:
      type: object
      required: [start, end]
      properties:
        start: { type: string, format: date-time }
        end: { type: string, format: date-time }
DeploymentAuthorization:
  type: object
  required: [id, release_id, authorized_by, authorized_at, window]
  properties:
    id: { type: string }
    release_id: { type: string }
    authorized_by: { type: string }
    authorized_at: { type: string, format: date-time }
    window: { type: object }
WebhookSubscription:
  type: object
  required: [callback_url, event_types]
  properties:
    id: { type: string }
    callback_url: { type: string, format: uri }
    event_types: { type: array, items: { type: string } }
    filter: { type: object }
    hmac_secret: { type: string, writeOnly: true }
    created_at: { type: string, format: date-time }
    active: { type: boolean }

```

Appendix B — Error Code Catalog

The following table enumerates every error ``type`` URI emitted by the Governance API. Each entry is a stable URI that resolves to a human-readable page. Clients SHALL treat unknown ``type`` URIs as the generic error of the same status code.

Type URI	Status	Meaning
https://gov-api.opuspublica.com/errors/unauthorized	401	Missing or invalid bearer token.
https://gov-api.opuspublica.com/errors/forbidden	403	Token lacks the required scope.
https://gov-api.opuspublica.com/errors/not-found	404	The requested resource does not exist.
https://gov-api.opuspublica.com/errors/validation	422	Schema validation failed; see ``errors`` array.
https://gov-api.opuspublica.com/errors/rate-limited	429	Per-scope rate limit exceeded; see ``Retry-After``.
https://gov-api.opuspublica.com/errors/idempotency-key-reuse	422	An ``Idempotency-Key`` was reused with a different request body.
https://gov-api.opuspublica.com/errors/constitution:invalid-filter	400	Invalid enum filter value.
https://gov-api.opuspublica.com/errors/constitution:not-found	404	Provision ID does not match any provision.
https://gov-api.opuspublica.com/errors/constitution:yaml-parse-error	422	Submitted YAML could not be parsed.
https://gov-api.opuspublica.com/errors/constitution:version-not-found	404	Named constitution version does not exist.
https://gov-api.opuspublica.com/errors/traceability:no-such-node	404	The ``start`` node does not exist.
https://gov-api.opuspublica.com/errors/traceability:cypher-rejected	422	Cypher query rejected by the safety filter.
https://gov-api.opuspublica.com/errors/traceability:no-chain	404	No chain exists for this requirement.
https://gov-api.opuspublica.com/errors/evidence:expired	200	Evidence exists but is stale (informational; returned with metadata).
https://gov-api.opuspublica.com/errors/evidence:edge-not-found	422	Specified edge ID does not exist; evidence stored but not attached.
https://gov-api.opuspublica.com/	422	Client-supplied hash does not match

errors/evidence:hash-mismatch		server-computed hash.
https://gov-api.opuspublica.com/ errors/evaluation:timeout	408	Rego rule did not complete within 5 seconds.
https://gov-api.opuspublica.com/ errors/evaluation:rego-error	500	Rego rule raised an error.
https://gov-api.opuspublica.com/ errors/signoff:gate-blocked	422	Cannot APPROVE: at least one GATE criterion FAILs.
https://gov-api.opuspublica.com/ errors/signoff:already-signed	409	A sign-off already exists for this release.
https://gov-api.opuspublica.com/ errors/remediation:already-resolved	409	Finding is already RESOLVED.
https://gov-api.opuspublica.com/ errors/slo:range-too-wide	400	Requested range exceeds 30 days.
https://gov-api.opuspublica.com/ errors/deploy:gate-not-passed	422	Cannot authorize: gate state is not PASSED.
https://gov-api.opuspublica.com/ errors/deploy:already-authorized	409	Release is already AUTHORIZED, DEPLOYED, or ROLLED_BACK.
https://gov-api.opuspublica.com/ errors/pagination:bad-cursor	400	Cursor is malformed or expired.
https://gov-api.opuspublica.com/ errors/internal	500	Unexpected internal error; see ``trace_id`` for diagnostics.
https://gov-api.opuspublica.com/ errors/unavailable	503	Service is temporarily unavailable; retry with backoff.

Appendix C — Scope-Permission Matrix

The following table maps each role to the scopes it grants and to the endpoints those scopes authorize. A blank cell means the role does not have the scope and cannot call any endpoint that requires it. The matrix is the normative reference for the governance administrator when binding service accounts to roles.

Scope	reader	operator	certifier	auditor	admin
constitution:read	yes	yes	yes	yes	yes
constitution:write					yes
traceability:read	yes	yes	yes	yes	yes
traceability:write		yes	yes		yes
certification:read	yes	yes	yes	yes	yes
certification:evaluate			yes		yes
certification:signoff			yes		yes
slo:read	yes	yes	yes	yes	yes
audit:read	yes	yes	yes	yes	yes
audit:write		yes	yes		yes
audit:export				yes	yes
release:read	yes	yes	yes	yes	yes
release:deploy		yes			yes
webhook:admin					yes
account:admin					yes

An endpoint that requires a scope not listed in the matrix returns ``403 Forbidden`` regardless of the role, because the scope is by definition not grantable. The ``operator`` role's ``release:deploy`` grant is restricted in practice to service accounts bound to the CSA office or to a designated deploy bot; the API enforces this restriction via an account attribute, not via the scope alone.

Appendix D — References

- OpenAPI 3.1.** OpenAPI Initiative. OpenAPI Specification 3.1.0. <https://swagger.io/specification/>
- RFC 9110.** Fielding, R., Reschke, J. HTTP Semantics, June 2022. <https://www.rfc-editor.org/rfc/rfc9110>
- RFC 7807.** Nottingham, M., Bryan, P. Problem Details for HTTP APIs, March 2016. <https://www.rfc-editor.org/rfc/rfc7807>
- RFC 8259.** Bray, T. The JavaScript Object Notation (JSON) Data Interchange Format, December 2017. <https://www.rfc-editor.org/rfc/rfc8259>
- RFC 6749.** Hardt, D. The OAuth 2.0 Authorization Framework, October 2012. <https://www.rfc-editor.org/rfc/rfc6749>
- RFC 7519.** Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT), May 2015. <https://www.rfc-editor.org/rfc/rfc7519>
- RFC 2119.** Bradner, S. Key words for use in RFCs to Indicate Requirement Levels, March 1997. <https://www.rfc-editor.org/rfc/rfc2119>
- RFC 3339.** Klyne, G., Newman, C. Date and Time on the Internet: Timestamps, July 2002. <https://www.rfc-editor.org/rfc/rfc3339>
- RFC 6902.** Bryan, P., Nottingham, M. JavaScript Object Notation (JSON) Patch, April 2013. <https://www.rfc-editor.org/rfc/rfc6902>
- RFC 4122.** Leach, P., Mealling, M., Salz, R. A Universally Unique Identifier (UUID) URN Namespace, July 2005. <https://www.rfc-editor.org/rfc/rfc4122>
- RFC 9745.** Nottingham, M. The Deprecation HTTP Response Header Field, February 2025. <https://www.rfc-editor.org/rfc/rfc9745>
- CloudEvents 1.0.** CNCF CloudEvents Specification 1.0.2. <https://cloudevents.io/>
- JSON Schema 2020-12.** JSON Schema Specification, Draft 2020-12. <https://json-schema.org/draft/2020-12/schema>
- OSCAL.** NIST OSCAL — Open Security Controls Assessment Language. <https://pages.nist.gov/OSCAL/>
- NIST SP 800-137.** NIST. Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations, September 2011. <https://csrc.nist.gov/pubs/sp/800/137/final>
- NIST SP 800-204C.** NIST. Implementation of DevSecOps for a Microservices Architecture with Service Mesh, July 2021. <https://csrc.nist.gov/pubs/sp/800/204c/iprd>
- OpenTelemetry.** CNCF. OpenTelemetry Specification. <https://opentelemetry.io/>
- Google SRE Workbook.** Google. Site Reliability Engineering: Workbook chapter on alerting on SLOs. <https://sre.google/workbook/>
- OPA / Rego.** Open Policy Agent (CNCf Graduated). Policy language and engine. <https://www.openpolicyagent.org/>
- Neo4j.** Neo4j property-graph database; Cypher query language. <https://neo4j.com/>
- Sigstore.** OpenSSF code-signing stack (Cosign, Rekor, Fulcio). <https://www.sigstore.dev/>
- CycloneDX.** OWASP-originated SBOM standard (ECMA-424). <https://cyclonedx.org/>
- SPDX.** Linux Foundation SBOM standard (ISO/IEC 5962:2021). <https://spdx.dev/>

SLSA. Supply-chain Levels for Software Artifacts (OpenSSF). <https://slsa.dev/>

Opus Publica RC2 Roadmap. Opus Publica. RC2 Evolution Roadmap v1.0. <https://opuspublica.com/> (internal)

— Issued under the authority of the Chief Systems Architect, Opus Publica — RC2 Governance API Specification —